



QuFiH: Hybrid low-bit quantization and block-level parameter efficient fine-tuning for video hashing

Fudong Li^{a,1}, Yuxiang Zhou^{b,1}, Qinglai Yang^b, Yong Chen^{b,*}, Dell Zhang^c, Xuelong Li^{c,d}

^a School of Mechanical and Electrical Engineering, Beijing Information Science and Technology University, Beijing, 100192, China

^b School of Computer Science, Beijing University of Posts and Telecommunications, Beijing, 100876, China

^c Institute of Artificial Intelligence (TeleAI), China Telecom, Shanghai, 200232, China

^d School of Artificial Intelligence, iOPEN, Northwestern Polytechnical University, Xi'an, 710129, China

ARTICLE INFO

Dataset link: <https://github.com/kydbj/QuFiH>

Keywords:
Learning to Hash
Video retrieval
Quantization
Multimodal IR

ABSTRACT

Current video hashing methods lack large-model-based deep feature representation capabilities, resulting in suboptimal retrieval performance on large-scale video databases. To address this, we propose Video-HLM (Video-Hashing Large Model), built upon the pretrained Video-LLaMA. However, its computational demands hinder practical deployment. To balance efficiency and performance, we develop a novel video hashing framework that fuses hybrid low-bit **Quantization** and block-level parameter-efficient **Fine-tuning** (QuFiH). Specifically, QuFiH employs **4-bit quantization for the foundation model** and **2-bit quantization for the hash head**, optimizing storage and performance. Additionally, we propose **block-level LoRA/Propulsion**, reducing redundant parameters while maintaining model expressiveness with lower computational overhead. Furthermore, we explore a “**distill-then-finetune**” strategy, combining knowledge distillation with the downstream task fine-tuning to enhance generalization and retrieval performance. Rich experiments on public datasets demonstrate QuFiH's superiority. It compresses model parameters by **6.84×** while outperforming state-of-the-art hashing methods in mAP@100, even surpassing the full-precision Video-HLM. QuFiH achieves **dual advancements in efficiency and accuracy**, offering a practical solution for large-scale video retrieval in resource-constrained environments. *Code is available at: <https://github.com/kydbj/QuFiH>.*

1. Introduction

In the digital era, video retrieval technology has been widely applied in various fields, such as the recommendation of social media content (Dong, Liu, Chen, Polak, & Zhang, 2024; Fan, Li, Liu, He, & Du, 2024), intelligent surveillance and behavior analysis (Duong, Le, & Hoang, 2023; Yang, Liu, Wu, Wu, & Liu, 2023), and medical imaging diagnosis (Daniel et al., 2023; Farhad et al., 2023). These applications require higher retrieval efficiency, lower storage costs, and optimized computational resources. Video hashing (Chen, 2023; Chen et al., 2025; Li, Tian, & Ng, 2024; Song, Yang, Huang, Shen, & Hong, 2011; Tolstikhin et al.,

* Corresponding author.

E-mail addresses: 201822238@bistu.edu.cn (F. Li), yuxiang.zhou@bupt.edu.cn (Y. Zhou), qinglai.y@bupt.edu.cn (Q. Yang), alphawolf.chen@gmail.com (Y. Chen), dell.z@ieee.org (D. Zhang), xuelong.li@ieee.org (X. Li).

¹ Fudong Li and Yuxiang Zhou are the co-first authors.

<https://doi.org/10.1016/j.ipm.2025.104408>

Received 2 May 2025; Received in revised form 5 September 2025; Accepted 6 September 2025

Available online 25 September 2025

0306-4573/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

2021; Xia, Ma, Liu, Mu, & Liu, 2017; Xu, 2017; Ye, Liu, Wang, & Chang, 2013) addresses these challenges by mapping high-dimensional video features into compact binary hash codes, significantly reducing storage requirements and accelerating similarity searches. It has emerged as a key technology in video retrieval. In recent years, machine learning methods based on pre-trained large models (GLM et al., 2024; Zhang, Li, & Bing, 2023) have demonstrated remarkable performance on various tasks. However, research on applying large models to video hashing remains unexplored. Therefore, we propose Video-HLM (Video-Hashing Large Model), built upon the multi-modal pre-trained Video-LLaMA (Zhang et al., 2023), to bridge this gap and harness the power of large-scale pre-training for enhanced video retrieval performance.

However, the high storage and computational costs of large models limit their deployment in resource-constrained environments, such as mobile devices and embedded systems (Liu, Galindo et al., 2024; Zheng et al., 2024; Zhu, Li, Liu, Ma, & Wang, 2024). Thus, a critical challenge in video retrieval is how to reduce resource consumption while maintaining the superior performance of large models. Quantization offers an effective way to reduce model complexity. However, traditional global quantization methods (Dettmers, Lewis, Belkada, & Zettlemoyer, 2022; Dettmers, Pagnoni, Holtzman, & Zettlemoyer, 2023; Frantar, Ashkboos, Hoefler, & Alistarh, 2022; Liu, Oguz et al., 2024; Yao et al., 2022) struggle to strike an ideal balance between performance and efficiency: low-bit quantization significantly reduces computational costs but leads to decreased model expressiveness, while high-bit quantization maintains performance but fails to fully optimize computational efficiency. Our research reveals that different network models in Video-HLM contribute differently to overall retrieval performance: the lower foundation model is responsible for extracting fine-grained feature information and requires higher precision, while upper hash model primarily handles feature transformation and semantic mapping functions, allowing for reduced precision to optimize computational efficiency. This finding aligns with human visual cognition mechanisms: in object recognition processes, preserving detailed information is crucial, while semantic generalization allows a certain degree of information loss. Based on this insight, we propose a hybrid low-bit quantization strategy for Video-HLM that applies quantization with different bits across models, achieving an optimized balance between computational efficiency and retrieval performance.

To further enhance the performance of the quantized model while controlling the scale of the parameters, this paper employs parameter-efficient fine-tuning techniques (such as LoRA Hu et al., 2022 and Propulsion Kowsheer, Prottasha, & Bhat, 2025) to optimize the quantized model. Existing methods predominantly adopt linear-level fine-tuning strategy, which add LoRA or Propulsion to linear modules (for clarity, we rename these as L-LoRA and L-Propulsion in this paper). Although these approaches provide significant flexibility, they may introduce redundant parameters. In this work, we systematically explore block-level fine-tuning, where LoRA or Propulsion is applied to one or multiple blocks, rather than linear module. We refer to this approach as B-LoRA and B-Propulsion. This marks the first systematic study of block-level fine-tuning in the context of video hashing, offering a novel solution to reducing computational costs while maintaining retrieval performance.

Furthermore, we innovatively integrate knowledge distillation (Beyer et al., 2022; Yang et al., 2022) with downstream task fine-tuning (Ding et al., 2023) to further enhance video hashing performance. Knowledge distillation improves the student model by transferring knowledge from a more powerful teacher model, while downstream task fine-tuning optimizes feature representation through task-specific loss functions. Although both techniques are effective individually, their synergistic interaction has yet to be thoroughly explored. In this work, we systematically explore the optimal combination of these two approaches, providing a new technical pathway for achieving efficient and high-precision video hash retrieval.

To sum up, we propose QuFiH, a novel video hashing framework based on hybrid low-bit (i.e., 2-bit for “Hash Model” and 4-bit for “Foundation Model” in Fig. 1) quantization and block-level parameter-efficient fine-tuning. Our main contributions are as follows:

- We are the first to construct a video hashing large model (Video-HLM), and innovatively adopt 4-bit quantization in the lower foundation model and 2-bit quantization in the upper hash model, achieving the optimal balance between storage efficiency and model performance. Experiments confirm that even with 2-bit quantization, our proposed method still maintains retrieval performance comparable to or even better than full-precision models.
- We propose a block-level fine-tuning strategy that applies LoRA/Propulsion at the Transformer block rather than for each linear module, effectively reducing redundant parameters while maintaining model expressiveness with lower computational overhead.
- We explore the optimal integration of knowledge distillation and downstream task fine-tuning, demonstrating that a “distill-then-finetune” strategy significantly enhances retrieval accuracy and recall.
- Experiments on ActivityNet and FCVID show that QuFiH outperforms existing hashing methods at 32-bit and 64-bit settings in mAP@100, even surpassing the full-precision Video-HLM.

The subsequent sections are structured in the following manner. We begin by reviewing prior work on video hashing and quantization, establishing the necessary background for our contributions. Section 3 presents research objects and statements. Section 4 details the proposed QuFiH method, including its key components and operational mechanisms. In Section 5, we benchmark QuFiH against state-of-the-art approaches and conduct ablation studies to evaluate the impact of its individual components. Section 6 further demonstrates QuFiH’s superior retrieval performance through a comprehensive video retrieval simulation. Finally, Sections 6.3 & 7 discuss the implications of our findings, outline potential applications, and suggest directions for future research.

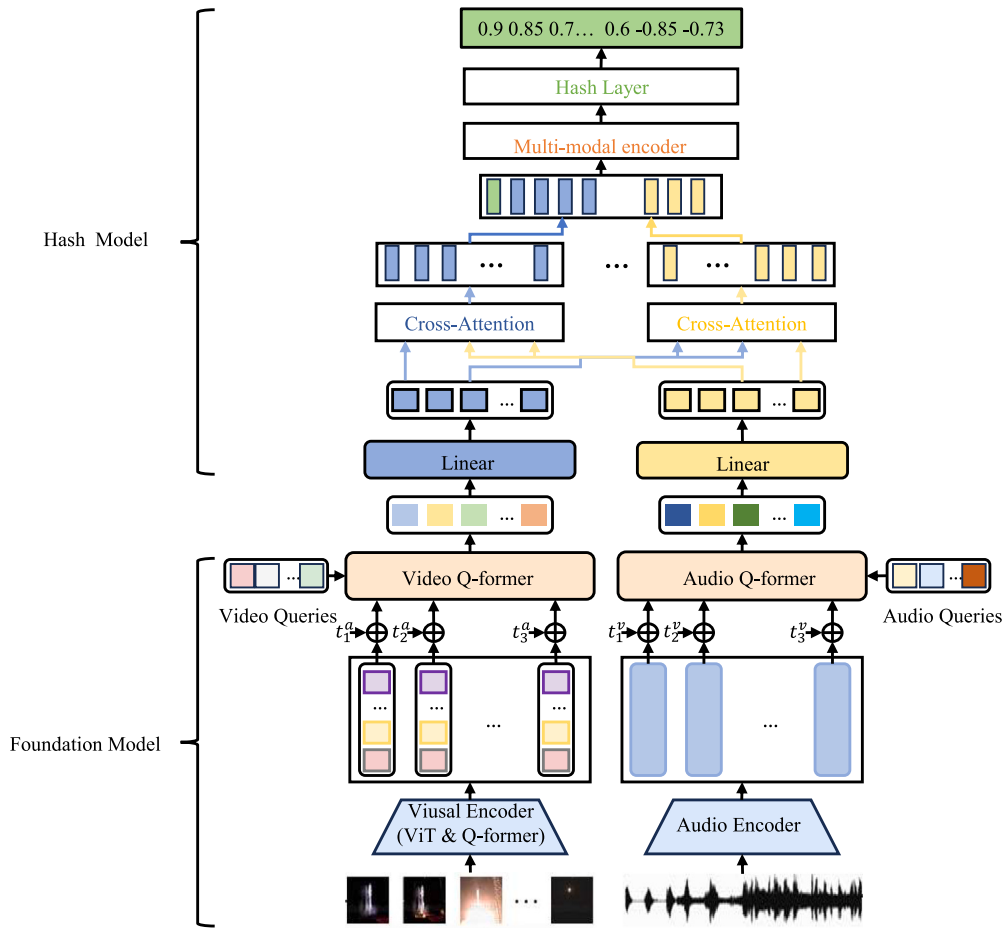


Fig. 1. The architecture of Video-HLM (Video-Hashing Large Model).

2. Related work

2.1. Video hashing

Video hashing (Chen, 2023; Chen et al., 2025; Li et al., 2024; Song et al., 2011; Tolstikhin et al., 2021; Xia et al., 2017; Xu, 2017; Ye et al., 2013) aims to generate semantic binary vectors for each video, significantly reducing storage space and improving computational speed. Early methods relied on pre-trained CNNs to extract features from video keyframes, averaging them to produce overall representations, and then using image hashing models (such as LSH Datar, Immorlica, Indyk, & Mirrokni, 2004, ITQ Gong, Lazebnik, Gordo, & Perronnin, 2013, SDH Li, Sun, He, & Tan, 2017; Shen, Shen, Liu, & Shen, 2015, COSDISH Kang, Li, & Zhou, 2016, SCDH Chen, Tian, Zhang, Wang, & Zhang, 2020, CSQ Yuan et al., 2020, LTH Chen, Hou et al., 2021 and MDSH Wang, Pan et al., 2023) to generate binary vectors as hash codes. Unlike the above approaches, DSVH (Chen, Hu et al., 2021) introduced 3DCNN for end-to-end spatiotemporal feature extraction and binary code generation. MLP-based methods (such as DVH Liong, Lu, Tan, & Zhou, 2017, MCMSH Hao et al., 2022 and EUVH Duan et al., 2024) achieve comprehensive video representation through various techniques. LSTM-based methods (including SRH Gu, Ma, & Yang, 2016, AVH Wang, Nie, Shi, Zhou, & Yin, 2021, SSTH Zhang, Wang, Hong, & Chua, 2016 and NPH Li, Chen, Lu, Li, & Zhou, 2019) model video structures to capture spatiotemporal content. Transformer-based models (such as ConMH Wang, Wang, Chen, Zeng and Xia, 2023, BTH Li, Li, Lu, & Zhou, 2021 and CHAIN Wei et al., 2023) utilize encoder-decoder architectures or contrastive learning frameworks for self-supervised binary code learning, with CHAIN also incorporating collaborative tasks to enhance spatiotemporal modeling capabilities.

Despite the rapid advancement of video hashing techniques, two critical research gaps remain unexplored. First, the integration of pre-trained large models with video hashing tasks has yet to be investigated. Second, quantization approaches for video hashing algorithms remain underdeveloped, with most methods operating exclusively at full precision. These gaps present promising new research directions: leveraging the cross-modal semantic capabilities of large models could significantly enhance the semantic expressiveness of video hash codes, while optimized quantization strategy could not only reduce computational and storage costs but potentially match or even exceed full-precision model performance in certain scenarios.

2.2. Quantization

With the increasing scale and complexity of deep learning models, efficient inference and deployment have become critical challenges. Model compression techniques (Hossain & Zhu, 2024; Li, Li and Meng, 2023; Park & Moon, 2025; Xu & McAuley, 2023; Zhu et al., 2024), have garnered significant attention, among which quantization (Dettmers et al., 2022, 2023; Frantar et al., 2022; Liu, Oguz et al., 2024; Yao et al., 2022) stands out as an effective approach. By converting high-precision weights and activations into lower-precision representations, quantization significantly reduces computational costs and memory footprint while enhancing inference efficiency and energy performance. Quantization methods are categorized into Quantization-Aware Training (QAT) and Post-Training Quantization (PTQ), each offering distinct trade-offs between accuracy and efficiency.

Quantization-Aware Training (QAT) maintains model performance by incorporating quantization into the training process. For example, LLM-QAT (Liu, Oguz et al., 2024) addresses the data limitations of QAT through data-free distillation, while QLoRA (Dettmers et al., 2023) introduces an efficient fine-tuning strategy that significantly reduces memory consumption. However, retraining large models is computationally expensive, requiring substantial GPU resources. As a more efficient alternative, Post-Training Quantization (PTQ) eliminates the need for retraining while achieving competitive performance. GPTQ (Frantar et al., 2022) leverages approximate second-order information to perform one-shot quantization, successfully reducing GPT-series models to 4-bit with minimal accuracy loss. ZeroQuant (Yao et al., 2022) employs a token-based quantization strategy to adapt to varying input distributions, though it relies on specialized CUDA kernel optimizations, limiting its applicability. Gpt3.int8() (Dettmers et al., 2022) combines INT8 and FP16 quantization to handle activation outliers, effectively constraining numerical ranges but potentially introducing computational overhead. SmoothQuant (Xiao, Lin, Seznec, Wu, Demouth, & Han, 2023) enhances 8-bit quantization by rescaling activations, reducing computational complexity and memory requirements. Meanwhile, AWQ (Lin et al., 2024) selectively preserves critical weights based on activation distributions, improving quantization accuracy while lowering computational costs.

Additionally, some studies integrate architectural compression with low-bit quantization to achieve higher compression ratios. PB-LLM (Yuan, Shang, & Dong, 2024) introduces partial binarization, selectively storing critical weights at higher precision. BiLLM (Huang et al., 2024) mitigates quantization loss by identifying important weights and applying a binary residual approximation strategy, while an optimal grouping method ensures both high-precision inference and computational efficiency.

However, existing PTQ methods typically apply fixed-bit quantization (e.g., 8-bit or 4-bit) uniformly across the entire model, failing to account for the varying sensitivity of different layers to quantization. Experiments show that, under the same bit width lower layers often suffer greater quantization errors than higher layers, potentially leading to significant information loss in critical components. To address this challenge, we propose a hybrid low-bit quantization strategy that balances computational efficiency, storage requirements, and model performance. Observations indicate that lower foundation model, responsible for extracting essential input features, is highly sensitive to quantization errors, which can significantly degrade performance. In contrast, higher hash model focus on semantic abstraction, making it more resilient to quantization noise. Based on this insight, we apply higher-bit quantization (e.g., 4-bit) to foundation model while using lower-bit quantization (e.g., 2-bit) for hash model, reducing computational overhead while preserving model expressiveness.

2.3. Distinction from existing work

While prior studies have made substantial progress in video hashing and model quantization, our work differs from existing approaches in two key aspects.

First, unlike traditional video hashing methods that primarily rely on CNNs, RNNs, or Transformer architectures, we explicitly explore the integration of large Multimodal models into the video hashing pipeline. This enables the transfer of rich cross-modal semantic knowledge, which has not been systematically investigated in earlier research.

Second, although quantization has been extensively studied in the context of language and vision models, its application to video hashing remains underexplored. Existing methods typically adopt uniform low-bit quantization across entire models, neglecting the heterogeneous sensitivity of different modules to quantization noise. In contrast, we propose a hybrid quantization strategy tailored to the video hashing framework, assigning higher precision to foundation layers responsible for feature extraction while applying lower precision to higher semantic hashing layers. This design achieves an improved balance between efficiency, storage, and retrieval accuracy.

Together, these contributions establish a novel paradigm that bridges large-model integration with quantization-aware video hashing, distinguishing our work from prior efforts in both domains.

3. The proposed method

3.1. Problem statement

In recent years, the paradigm of leveraging pretrained large models as a foundation model while constructing task-specific downstream modules has become mainstream (Pan, Lin, Zhu, Shao, & Li, 2022; Sung, Cho, & Bansal, 2022; Wang, Panda et al., 2023). This approach fully exploits the rich feature representations of large models, leading to significant performance improvements across various machine learning tasks. In this context, we introduce Video-HLM (Video Hashing Large Model), a pioneering video hashing framework built upon a pretrained large model. However, the high storage and computational costs of large models pose challenges for deployment in resource-constrained environments (e.g., mobile devices, embedded systems). Thus, a key challenge in

video retrieval is how to maintain the strong performance of Video-HLM while significantly reducing its storage and computational overhead. To address this, we propose a hybrid low-bit quantization strategy that effectively preserves model performance while substantially reducing storage and inference costs.

Given a dataset containing V videos, denoted as $S = \{v_i\}_{i=1}^V$, the objective of the proposed video hashing method is to design an efficient quantized hashing function that maps each video v_i to a q -bit binary code $\mathbf{b}_i$.

To fully utilize the multimodal information in videos, each video is first preprocessed to extract N keyframes and corresponding audio segments. As illustrated in Fig. 1, the pretrained Video-LLaMA (Zhang et al., 2023) is utilized, where its “visual branch” and “audio branch” are responsible for extracting feature representations from keyframes and audio segments, respectively. We collectively refer to the visual and audio branches as the “**Foundation Model**”. Next, these multimodal features are fed into the “**Hash Model**”. Within this model, each modality-specific branch employs a cross-attention mechanism to capture interactions between different modalities, after which the extracted features are concatenated. The resulting multimodal representations are then passed through a Transformer-based encoder to capture high-level semantic information. This embedding is then binarized into the final hash code $\mathbf{b}_i \in \{-1, +1\}^q$ using a $\text{Sign}(\cdot)$ function for video v_i , where q represents the code length.

A key objective of this study is to significantly reduce storage and computational costs through quantization while maintaining model accuracy. To achieve this, we propose a hybrid low-bit quantization strategy to quantize the two components of Video-HLM: **Foundation Model** and **Hash Model**. The **Foundation Model**, responsible for feature extraction, contains a large number of parameters and is quantized using NF4 (Dettmers et al., 2023) to preserve strong representation capabilities, while the **Hash Model**, which generates hash codes with fewer parameters, is further compressed using NF2. After quantization, we introduce a block-level LoRA/Propulsion strategy that adapts at the Transformer block level rather than modifying each linear module, reducing adaptation complexity and improving efficiency. Finally, by incorporating knowledge distillation and downstream fine-tuning, we further enhance the performance of the quantized model, resulting in **QuFiH**, which achieves an optimal balance between computational efficiency and retrieval accuracy.

3.2. Model architecture

The Video-HLM consists of two main components: the “Foundation Model” and the “Hash Model”. The “Foundation Model” includes the “Input” and “Modality-Specific Encoder”. The “Hash Model” comprises the “Cross-Modal Attention”, the “Multi-Modal Encoder”, and the “Hash Layer” (see Fig. 1).

Input. The model input consists of a sequence of N video keyframes and their corresponding N audio segments, with each audio segment aligned to a specific keyframe.

Modality-Specific Encoder. To process multimodal data, Video-HLM leverages Video-LLaMA’s (Zhang et al., 2023) visual and audio branches to extract features from images and audio, respectively.

In the visual branch, given a sequence of N video keyframes, we utilize the pretrained image encoder from Video-LLaMA (Zhang et al., 2023) to project each frame into a set of K_f image embedding vectors, forming the video frame representation:

$$\mathbf{I} = [\mathbf{I}_1, \mathbf{I}_2, \dots, \mathbf{I}_N], \quad (1)$$

where $\mathbf{I}_i \in \mathbb{R}^{K_f \times d_f}$ represents the set of K_f image embeddings for the i th frame, each of dimension d_f . To incorporate sequential information, the model applies a positional embedding layer, adding positional encodings to each frame representation:

$$\mathbf{I}'_i = \mathbf{I}_i + \text{PE}_I(i), \quad (2)$$

where $\text{PE}_I(i) \in \mathbb{R}^{K_f \times d_f}$ denotes the positional encoding for the i th frame. Subsequently, the position-encoded frame representations are fed into a Video Q-Former, which aggregates these embeddings (following a similar structure to the Q-Former in BLIP-2 Li, Li, Savarese and Hoi, 2023) and generates a fixed-length video feature representation:

$$\hat{\mathbf{I}} = \text{Video Q-Former}(\mathbf{I}'_1, \mathbf{I}'_2, \dots, \mathbf{I}'_N : \text{Video Queries}), \quad (3)$$

where $\hat{\mathbf{I}} \in \mathbb{R}^{K_f \times d_I}$, representing a set of K_f video embedding vectors, each with a dimension of d_I . “Video Queries” are a fixed set of pre-defined, learnable vectors. Through interaction with image tokens from the visual encoder via the Video Q-Former, they aggregate visual information. Each query token captures a distinct visual concept—such as specific objects, attributes, or background elements. For further details, please refer to Zhang et al. (2023). To enable interaction with the LLM in a shared semantic space, the video features are projected using a linear transformation:

$$\mathbf{z}_I = \hat{\mathbf{I}} \times \mathbf{W}_I + \mathbf{b}_I, \quad (4)$$

where $\mathbf{W}_I \in \mathbb{R}^{d_I \times d_{\text{LLM}}}$ represents the projection matrix, and $\mathbf{b}_I$ is the bias. The resulting $\mathbf{z}_I \in \mathbb{R}^{K_f \times d_{\text{LLM}}}$ represents the final input features for the image modality.

In the audio branch, the video is first uniformly segmented into N audio clips, which are then converted into 128-dimensional Mel spectrograms. A pretrained audio encoder (e.g., ImageBind Girdhar et al., 2023) is then used to map each spectrogram into a dense feature vector, obtaining the audio representation:

$$\mathbf{A} = [\mathbf{A}_1, \mathbf{A}_2, \dots, \mathbf{A}_N], \quad (5)$$

where $\mathbf{A}_i \in \mathbb{R}^{d_A}$ represents the feature of the i th audio clip. To incorporate temporal information, a positional embedding layer is applied:

$$\mathbf{A}'_i = \mathbf{A}_i + \text{PE}_A(i). \quad (6)$$

Next, an Audio Q-Former aggregates the clip-level features to generate a fixed-length representation:

$$\hat{\mathbf{A}} = \text{Audio Q-Former}(\mathbf{A}'_1, \mathbf{A}'_2, \dots, \mathbf{A}'_N : \text{Audio Queries}). \quad (7)$$

Here, $\hat{\mathbf{A}} \in \mathbb{R}^{K_A \times d_A}$, where K_A is the number of embedded audio tokens, and d_A is the dimension of each embedding. Similar to “Video Queries”, the “Audio Queries” are also a fixed set of pre-defined, learnable vectors. Through the Audio Q-Former, they interact with audio token representations and produce output tokens that encode core acoustic concepts (Zhang et al., 2023). Finally, the resulting audio representation $\hat{\mathbf{A}}$ is projected into the shared embedding space of the LLM’s text representation via a linear transformation, yielding the final audio feature embeddings:

$$\mathbf{z}_A = \hat{\mathbf{A}} \times \mathbf{W}_A + \mathbf{b}_A, \quad (8)$$

where $\mathbf{W}_A \in \mathbb{R}^{d_A \times d_{\text{LLM}}}$ is the projection matrix, and $\mathbf{b}_A$ is the bias. $\mathbf{z}_A \in \mathbb{R}^{K_A \times d_{\text{LLM}}}$ serves as the input feature representation for the audio modality.

Cross-Modal Attention. After unimodal feature encoding, we employ a cross attention to capture the semantic interactions between audio and visual modalities. Concretely, we use a multi-head cross-attention (MCA) with a linear fully connected (FC) layer:

$$\mathbf{z}_{m_2 \leftarrow m_1} = \text{FC}(\text{MCA}(\mathbf{z}_{m_1}, \mathbf{z}_{m_2})) + \mathbf{z}_{m_1}, \quad (9)$$

where m_1 denotes the target modality and m_2 is the source modality, and the MCA is given as follow:

$$\text{MCA}(\mathbf{z}_{m_1}, \mathbf{z}_{m_2}) = \text{Concat}(\text{head}_1, \dots, \text{head}_{A_c})\mathbf{W}, \quad (10)$$

$$\text{head}_i = \text{Softmax}\left(\frac{\mathbf{Q}_i^{m_1} \times \mathbf{K}_i^{m_2 T}}{\sqrt{d}}\right) \mathbf{V}_i^{m_2}, i = 1, \dots, A_c, \quad (11)$$

and

$$\mathbf{Q}_i^m = \mathbf{z}_m \times \mathbf{W}_i^Q, \quad (12)$$

$$\mathbf{K}_i^m = \mathbf{z}_m \times \mathbf{W}_i^K, \quad (13)$$

$$\mathbf{V}_i^m = \mathbf{z}_m \times \mathbf{W}_i^V, \quad (14)$$

where $\mathbf{W}$, $\mathbf{W}_i^Q$, $\mathbf{W}_i^K$, and $\mathbf{W}_i^V$ are learnable matrices, and A_c equals to $A_{A \rightarrow I}$ or $A_{I \rightarrow A}$. Thus, we obtain representations enriched with cross-modal information $\mathbf{z}_{A \rightarrow I}$, $\mathbf{z}_{I \rightarrow A}$, by the following formula:

$$\mathbf{z}_{A \leftarrow I} = \text{FC}(\text{MCA}(\mathbf{z}_I, \mathbf{z}_A)) + \mathbf{z}_I, \quad (15)$$

$$\mathbf{z}_{I \leftarrow A} = \text{FC}(\text{MCA}(\mathbf{z}_A, \mathbf{z}_I)) + \mathbf{z}_A. \quad (16)$$

Finally, we concatenate the representations from both modalities to obtain the fused feature:

$$\mathbf{z}_{AI} = [\mathbf{z}_{I \leftarrow A}, \mathbf{z}_{A \leftarrow I}]. \quad (17)$$

Multi-Modal Encoder. After concatenating the features from both modalities, we introduce a multimodal Transformer encoder to obtain the final video representation. This encoder consists of L_M Transformer blocks, with each composed of Multi-head Self-Attention (MSA) and an MLP. Before feeding $\mathbf{z}_{AI}$, a **cls** token is added at the beginning of $\mathbf{z}_{AI}$, and its corresponding output serves as the video’s final representation:

$$\mathbf{z}_V^0 = [\text{cls}^{AI}; \mathbf{z}_{AI}] + \mathbf{E}_{\text{pos}}^{AI}, \quad (18)$$

$$\mathbf{z}_V^{\ell'} = \text{MSA}(\text{LN}(\mathbf{z}_V^{\ell-1})) + \mathbf{z}_V^{\ell-1}, \quad (19)$$

$$\mathbf{z}_V^{\ell} = \text{MLP}(\text{LN}(\mathbf{z}_V^{\ell'})) + \mathbf{z}_V^{\ell'}, \quad (20)$$

where $\ell = 1, \dots, L_M$. By following the above procedure, we can get the latent representation $\mathbf{z}_V^{L_M}$ of video $\mathbf{v}_i$. Note that the output $\mathbf{z}_V^{L_M}$ of each encoder also contains $(1 + N)$ parts corresponding to the preset token CLS and N multimodal features. We employ the first part’s embedding, i.e., $\mathbf{z}_V^{L_M}[0]$, as its final representation.

Hash Layer. To obtain the final hash codes of video $\mathbf{v}_i$, we add a hash layer, i.e., a FC network with the $\text{Tanh}(\cdot)$ function, on top of the Multi-Modal Encoder. Formally, given video $\mathbf{v}_i$, we transform its Multi-Modal Encoder's output $\mathbf{z}_V^{LM}[0]$ into a q -dimensional binary-like real-valued vector $\mathbf{g}_i$ via:

$$\mathbf{g}_i = \text{Tanh}(\text{FC}(\mathbf{z}_V^{LM}[0])) \in \mathbb{R}^q, \quad (21)$$

which is then forwarded to the $\text{Sign}(\cdot)^2$ function for the final binary code $\mathbf{b}_i$, i.e.,

$$\mathbf{b}_i = \text{Sign}(\mathbf{g}_i) \in \{-1, +1\}^q. \quad (22)$$

3.3. Hybrid low-bit quantization

In this study, we employ NF4 (Dettmers et al., 2023) and NF2 quantization methods to reduce model storage requirements and computational overhead while minimizing quantization errors. This approach ensures efficient model performance and strong representational capability. Below, we provide a detailed explanation of the NF4 and NF2 quantization methods and describe the quantization strategy used in the model.

Quantization Precision: NF4 and NF2 Quantization. NF4 (4-bit NormalFloat) is an optimized data type based on quantile quantization. It aims to ensure equal data points in each quantization bin, minimizing information loss. The process involves computing the empirical cumulative distribution function (ECDF) of the input tensor and estimating 17 quantile points to divide the data into 16 intervals. However, approximate quantile estimation algorithms used for computational efficiency may introduce significant quantization errors, particularly in the presence of outliers. To address the aforementioned issue, NF4 introduces a more efficient solution. It assumes that neural network weights follow a standard normal distribution with zero mean and standard deviation σ . By scaling σ , the parameter distribution can be effectively mapped to the $[-1, 1]$, aligning with the quantization data type constraints. The theoretical normal distribution $\mathcal{N}(0, 1)$ is divided into 17 quantile points, forming 16 discrete quantization values, defined as:

$$q_i = \frac{1}{2} \left(Q_X \left(\frac{i}{2^k + 1} \right) + Q_X \left(\frac{i+1}{2^k + 1} \right) \right), \quad (23)$$

where $Q_X(\cdot)$ represents the quantile function of the standard normal distribution, q_i represents discrete quantization value. This approach effectively optimizes the quantization process, reducing quantization errors and improving the efficiency of the NF4 quantization method.

NF2 is an extreme compression variant of NF4, following the same fundamental concept but further reducing the number of quantization levels. In NF2, only 4 quantization levels (2-bit) are used, defined as:

$$\mathcal{Q}^{\text{NF2}} = \{q_1, q_2, q_3, q_4\}. \quad (24)$$

Since NF2 has fewer quantization levels, it introduces higher quantization errors. However, it significantly reduces storage requirements and computational overhead making it highly suitable for low-complexity computations in the final stages of model inference, such as post-Huffman decoding.

Hybrid Low-Bit Quantization. To achieve an optimal balance between model compression and performance preservation, we propose a hybrid low-bit quantization strategy. For the **Foundation Model**, which contains the majority of parameters, we employ NF4 quantization to maintain the strong representational capabilities of the foundation model while effectively reducing quantization errors. For the **Hash Model**, which has relatively fewer parameters, a more aggressive NF2 quantization method is applied to further compress the model size. This hybrid low-bit quantization strategy fully considers the characteristics and roles of different parts of the model. By applying higher-precision quantization (NF4) in **Foundation Model**, critical feature information is preserved, while lower-precision quantization (NF2) in **Hash Model** further reduces model size. This approach effectively lowers storage and computational costs while maximally maintaining model performance.

For the **Foundation Model**, we apply NF4 quantization, formulated as:

$$\mathbf{W}^{\text{NF4}} = \text{quant}_{\text{NF4}}(\mathbf{W}) = \arg \min_{q_i} |\mathbf{W} - q_i|, \quad (25)$$

where q_i represents the 16 predefined quantization centers of NF4. During computation, the quantized weights $\mathbf{W}^{\text{NF4}}$ should be dequantized. Since NF4 is based on quantile quantization, a predefined lookup table (LUT) can be used to efficiently map $\mathbf{W}^{\text{NF4}}$ back to floating-point values:

$$\mathbf{W} = \text{dequant}_{\text{NF4}}(\mathbf{W}^{\text{NF4}}) = \text{LUT}(\mathbf{W}^{\text{NF4}}). \quad (26)$$

The $\text{LUT}(\cdot)$ restores $\mathbf{W}^{\text{NF4}}$ to an approximate floating-point representation, ensuring efficient weight recovery.

For the **Hash Model**, NF2 quantization is applied, and its quantization and dequantization methods are similar to those of NF4.

Consider the entire model where the **Foundation Model** parameters are denoted as θ and the **Hash Model** parameters as μ . These parameters are quantized using NF4 and NF2 quantization schemes:

$$\theta^{\text{NF4}} = \text{quant}_{\text{NF4}}(\theta), \quad (27)$$

² For a given input x , the sign function $\text{Sign}(x)$ is defined as $+1$ if $x \geq 0$, and -1 otherwise. For a vector input $\mathbf{x}$, the function is applied element-wise.

$$\mu^{\text{NF2}} = \text{quant}_{\text{NF2}}(\mu). \quad (28)$$

For a video consisting of N keyframes $\{\mathbf{I}_j\}_{j=1}^N$ and N audio segments $\{\mathbf{A}_j\}_{j=1}^N$, the forward propagation process is formulated as:

$$(\mathbf{z}_I, \mathbf{z}_A) = \mathbf{f}_{\text{dequant}_{\text{NF4}}(\theta^{\text{NF4}})} \left(\{\mathbf{I}_j\}_{j=1}^N, \{\mathbf{A}_j\}_{j=1}^N \right), \quad (29)$$

$$\mathbf{b} = \mathbf{h}_{\text{dequant}_{\text{NF2}}(\mu^{\text{NF2}})}(\mathbf{z}_I, \mathbf{z}_A), \quad (30)$$

where $\mathbf{f}(\cdot)$ represents the **Foundation Model**, and $\mathbf{z}_I, \mathbf{z}_A$ are the visual and audio features, respectively. The function $\mathbf{h}(\cdot)$ denotes the **Hash Model**, responsible for computing the hash features.

During forward propagation, the quantized parameters θ^{NF4} and μ^{NF2} are first dequantized to floating-point values before being used. This approach significantly reduces the size of the model during inference, while maintaining accuracy comparable to that of the full-precision model.

3.4. Quantized model training

3.4.1. Fine-tuning techniques for quantized models

To mitigate potential performance degradation from quantization, we leverage two parameter-efficient fine-tuning approaches: LoRA (Hu et al., 2022) and Propulsion (Kowsher et al., 2025). These techniques effectively augment the model's expressive power while preserving computational and memory efficiency, ensuring robust performance in the quantized regime.

LoRA and Propulsion (L-LoRA and L-Propulsion). LoRA reduces training overhead by introducing a low-rank matrix to each linear module and updating only this matrix during fine-tuning. Given a standard linear transformation:

$$\mathbf{Y} = \mathbf{X} \times \mathbf{W} + \mathbf{b}, \quad (31)$$

where $\mathbf{X} \in \mathbb{R}^{s \times d_{in}}$ is the input feature matrix, $\mathbf{W} \in \mathbb{R}^{d_{in} \times d_{out}}$ is the weight matrix, $\mathbf{b} \in \mathbb{R}^{d_{out}}$ is the bias term, $\mathbf{Y} \in \mathbb{R}^{s \times d_{out}}$ is the output feature matrix, s is the sequence length, and d_{in} is the input feature dimension. In LoRA, two low-rank matrices, $\mathbf{A}$ and $\mathbf{B}$, are introduced, modifying the linear transformation to:

$$\mathbf{Y} = \mathbf{X} \times \mathbf{W} + \mathbf{X} \times \mathbf{B} \times \mathbf{A} + \mathbf{b}, \quad (32)$$

where $\mathbf{A} \in \mathbb{R}^{r \times d_{out}}$ and $\mathbf{B} \in \mathbb{R}^{d_{in} \times r}$ are low-rank matrices with rank $r \ll \min\{d_{in}, d_{out}\}$. During fine-tuning, only $\mathbf{A}$ and $\mathbf{B}$ are updated, significantly reducing computational cost.

Propulsion enhances the representational capacity of linear modules by introducing a trainable scaling matrix to adjust their outputs. Unlike LoRA, which directly adds low-rank matrices to linear module, Propulsion employs an element-wise scaling mechanism. Given the output $\mathbf{Y} \in \mathbb{R}^{s \times d_{out}}$, Propulsion introduces a trainable scaling vector:

$$\mathbf{p} = \{p_1, p_2, \dots, p_{d_{out}}\} \in \mathbb{R}^{d_{out}}. \quad (33)$$

For a given token representation $\mathbf{y}_j$ in $\mathbf{Y}$:

$$\mathbf{y}_j = \{y_1, y_2, \dots, y_{d_{out}}\}, \quad (34)$$

Propulsion applies an element-wise transformation:

$$\mathbf{y}'_j = \mathbf{y}_j \odot \mathbf{p}, \quad (35)$$

which expands to:

$$\mathbf{y}'_j = [y_1 \cdot p_1, y_2 \cdot p_2, \dots, y_{d_{out}} \cdot p_{d_{out}}], \quad (36)$$

where $\odot$ denotes element-wise multiplication. Extending this transformation to the entire output $\mathbf{Y}$, the Propulsion-modified output $\mathbf{Y}'$ is given by:

$$\mathbf{Y}' = [\mathbf{y}_1 \odot \mathbf{p}, \mathbf{y}_2 \odot \mathbf{p}, \dots, \mathbf{y}_s \odot \mathbf{p}]. \quad (37)$$

Each token representation is adaptively scaled by $\mathbf{p}$, allowing the linear module's output to better align with task-specific requirements. To enhance flexibility, Propulsion further introduces a polynomial scaling transformation, where the scaling vector $\mathbf{p}$ undergoes an exponentiation:

$$\mathbf{Y}' = [\mathbf{y}_1 \odot \mathbf{p}^k, \mathbf{y}_2 \odot \mathbf{p}^k, \dots, \mathbf{y}_s \odot \mathbf{p}^k], \quad (38)$$

where the hyperparameter k controls the nonlinearity of the transformation.

In our approach, we integrate either a LoRA or Propulsion module into each linear module to enhance adaptability. To distinguish between them, we rename them as L-LoRA and L-Propulsion.

Block-Level LoRA and Block-Level Propulsion (B-LoRA and B-Propulsion). Traditional LoRA and Propulsion methods introduce fine-tuning modules into each linear module. Since a Transformer architecture consists of multiple linear modules (e.g., Query, Key, and Value, Feed-Forward Network (FFN), etc.), adding a separate LoRA or Propulsion module to each linear

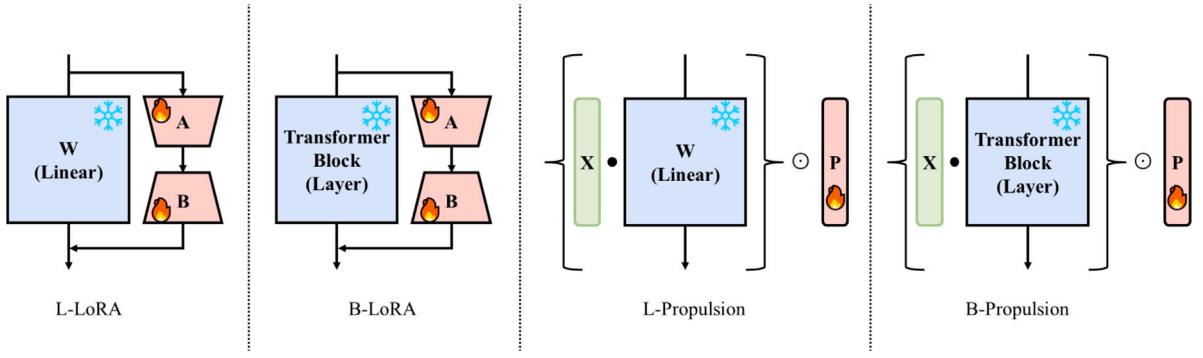


Fig. 2. Four fine-tuning methods: L-LoRA, B-LoRA, L-Propulsion, and B-Propulsion.

module results in a significant increase in parameters and computational cost. To address this, we propose block-level LoRA (B-LoRA) and block-level Propulsion (B-Propulsion), where fine-tuning modules are applied at the Transformer block level instead of the linear level. This significantly reduces parameter overhead while maintaining adaptability.

Consider a pre-trained Transformer model with M blocks:

$$\mathcal{B} = \{B_1, B_2, \dots, B_M\}. \quad (39)$$

Let $\mathbf{h}^{(i-1)}$ be the input to the i th Transformer block. The block-level LoRA transformation is defined as:

$$\mathbf{h}^{(i)} = B_i(\mathbf{h}^{(i-1)}; \mathbf{W}^i) + \mathbf{h}^{(i-1)} \mathbf{B}^i \mathbf{A}^i, \quad (40)$$

where $\mathbf{W}^i$ represents the frozen pre-trained weights of the i th Transformer block, $\mathbf{h}^{(i)}$ is the output of the i th block, $\mathbf{A}^i$ and $\mathbf{B}^i$ are low-rank matrices introduced in the i th Transformer block.

The block-level Propulsion transformation is formulated as:

$$\mathbf{h}^{(i)} = B_i(\mathbf{h}^{(i-1)}; \mathbf{W}^i) \odot (\mathbf{p}^i)^k, \quad (41)$$

where $\mathbf{p}^i \in \mathbb{R}^{d_{\text{out}}}$ is the trainable Propulsion scaling vector for the i th Transformer block, $\odot$ denotes element-wise multiplication, k is a hyperparameter controlling the nonlinearity of the transformation.

Fig. 2 illustrates the differences between linear-level and block-level LoRA and Propulsion.

3.4.2. Loss function

Knowledge Distillation. To mitigate the accuracy loss caused by quantization, we employ knowledge distillation, where the original unquantized model serves as the teacher model, and the quantized LoRA/Propulsion-adapted model acts as the student model. The goal is to train the student model to mimic the teacher model's behavior, thereby reducing the discrepancy between them in the feature space.

During distillation, we align not only the final output features of the teacher and student models but also their intermediate hidden state representations. Let $\mathbf{h}_T^j$ and $\mathbf{h}_S^j$ denote the hidden states of the teacher and student models at the j th component, respectively, and let $\mathbf{o}_T$ and $\mathbf{o}_S$ represent their final outputs. The distillation loss is defined as:

$$\mathcal{L}_{\text{distill}} = \sum_{j=1}^c \alpha_j \cdot D(\mathbf{h}_T^j, \mathbf{h}_S^j) + \beta \cdot D(\mathbf{o}_T, \mathbf{o}_S), \quad (42)$$

where α_j is the weight for the j th component output, β is the alignment weight for the final output, $D(\cdot, \cdot)$ represents a distance metric, c denotes the number of components in the hash model, which is the only part that requires fine-tuning. In our experimental setup, we fixed α_j to 1 and β to 2.

In our approach, we use the Mean Squared Error (MSE) as the distance function:

$$D_{\text{MSE}}(\mathbf{x}, \mathbf{y}) = \frac{1}{n} \sum_{i=1}^n (x_i - y_i)^2, \quad (43)$$

where $\mathbf{x} = [x_1, \dots, x_n]$ and $\mathbf{y} = [y_1, \dots, y_n]$ are vectors of equal length, and n is the dimensionality of the vectors.

By enforcing alignment at multiple levels, the student model not only learns to replicate the teacher model's final predictions but also maintains consistency across different feature representations, ultimately improving fine-tuning performance.

Downstream Tasks Optimization. In hash learning tasks, the objective is to ensure that similar samples have similar hash codes in the hash space, while dissimilar samples are mapped far apart. To achieve this, the InfoNCE loss is employed to optimize hash codes, enabling the model to better capture the underlying similarity structure in the data.

The InfoNCE loss consists of three types of samples: an anchor sample $\mathbf{a}$, a positive sample $\mathbf{p}$, and n_o negative samples $\{\mathbf{n}_i\}_{i=1}^{n_o}$. The anchor and positive samples belong to the same class, while the anchor and negative samples belong to different classes. The InfoNCE loss maximizes the agreement between positive pairs while minimizing similarity between anchor-negative pairs. It is defined as:

$$\mathcal{L}_{\text{InfoNCE}}(\mathbf{a}, \mathbf{p}, \mathbf{n}) = -\log \frac{\exp(\text{sim}(\mathbf{a}, \mathbf{p})/\tau)}{\sum_{i=1}^{n_o} \exp(\text{sim}(\mathbf{a}, \mathbf{n}_i)/\tau) + \exp(\text{sim}(\mathbf{a}, \mathbf{p})/\tau)}, \quad (44)$$

where $\mathbf{a}$, $\mathbf{p}$, and $\mathbf{n}$ represent the embedding vectors of the anchor, positive, and negative samples, respectively.³ The function $\text{sim}(\cdot, \cdot)$ denotes a similarity measure (e.g., cosine similarity), and τ is a temperature parameter controlling the discrimination of negative samples.

Overall Loss Function. In practice, we explore two strategies for integrating the distillation loss and the InfoNCE loss:

(1) Weighted-Sum. In this approach, the overall loss is formulated as a weighted sum of the distillation loss and the InfoNCE loss:

$$\mathcal{L} = \alpha \cdot \mathcal{L}_{\text{distill}} + (1 - \alpha) \cdot \mathcal{L}_{\text{InfoNCE}}, \quad (45)$$

where α is a hyperparameter that balances the contribution of the two losses. This method allows simultaneous optimization of task adaptability and hash code quality.

(2) Two-Stage Training. In this approach, training is performed in two sequential stages. First, the model is fine-tuned using the distillation loss to ensure the student model closely mimics the teacher model. Then, the InfoNCE loss is applied to optimize the downstream task:

$$\mathcal{L} = \begin{cases} \mathcal{L}_{\text{distill}}, & \text{Distillation Stage,} \\ \mathcal{L}_{\text{InfoNCE}}, & \text{Downstream Fine-Tuning Stage;} \end{cases} \quad (46)$$

this strategy provides more stable training but requires a little longer training time.

3.4.3. Training pipeline

This part presents a detailed overview of the QuFiH training pipeline, which comprises three key stages: (1) full-precision pretraining, (2) quantization and adaptation, and (3) fine-tuning. The overall process is designed to maintain high retrieval performance while significantly improving computational efficiency through quantization, knowledge distillation, and downstream task optimization. The process is divided into three key stages:

1. Full-Precision Pretraining (Stage 1 in Algorithm 1). In this stage, a full-precision teacher model (Video-HLM) is trained. The foundation model is initialized with the pretrained Video-LLaMA (Zhang et al., 2023) and remains frozen during training, while the upper hash model are randomly initialized and optimized. The objective of this stage is to learn hash codes using the InfoNCE loss function, ensuring the model captures meaningful semantic representations.
2. Quantization and Adaptation (Stage 2 in Algorithm 1). Following pretraining, both the foundation and hash models are quantized: the foundation model is converted to NF4 format and remains frozen, while the hash model is quantized into NF2 format. To compensate for performance degradation due to quantization, lightweight adaptation modules—such as L-LoRA, L-Propulsion, B-LoRA, or B-Propulsion—are inserted into the hash model. Only the parameters of these adaptation modules are updated during this stage, forming the quantized student model (QuFiH).
3. Fine-Tuning of the Quantized Model (Stage 3 in Algorithm 1). In the final stage, knowledge distillation is employed to align both the intermediate representations and output distributions of the student model (QuFiH) with those of the full-precision teacher (Video-HLM). Meanwhile, the InfoNCE loss continues to guide the optimization of the hash codes for enhanced downstream retrieval performance. Two fine-tuning strategies are explored, i.e., Strategy 1: Joint Optimization and Strategy 2: Two-Stage Training, as outlined in Algorithm 1.

To sum up, as shown in Algorithm 1, the training of QuFiH is divided into three stages; through these stages, it substantially reduces storage and computational costs while maintaining—or even exceeding—the video hashing retrieval performance of the full-precision Video-HLM model from the first stage.

4. Experiments

4.1. Datasets

We evaluate our QuFiH using two large-scale benchmark video datasets: ActivityNet (Heilbron, Escorcia, Ghanem, & Niebles, 2015) and FCVID (Jiang, Wu, Wang, Xue, & Chang, 2018).

ActivityNet consists of a wide range of human activities annotated into 200 categories. We randomly sampled 10,023 videos as the training set and retrieval database, with 2512 and 2413 videos as the validation and testing sets, respectively.

FCVID consists of 91,016 videos annotated into 239 categories. The dataset is split into training, validation, and testing sets, corresponding to 45,508, 22,754, and 22,754 videos, respectively. Additionally, the training set is also employed as the video database for retrieval.

³ For example, given a randomly sampled video $\mathbf{a}$ from a specific class, any other video $\mathbf{p}$ from the same class is treated as a positive sample, while videos $\{\mathbf{n}_i\}_{i=1}^{n_o}$ sampled from different classes are considered negative samples for $\mathbf{a}$.

Algorithm 1 Hybrid Low-Bit Quantization and Block-Level Parameter Efficient Fine-Tuning for Video Hashing

```

1: Input: Training dataset  $S$ , pretraining epochs  $T_s$ , fine-tuning epochs  $T_f$ ;
2: Output: Fine-tuned quantized student model QuFiH.
3: /*———— Stage 1: Full-Precision Pretraining ———*/
4: Initialize the teacher model Video-HLM:
5:   Foundation Model:  $F(\cdot; \theta)$ , initialized with pretrained Video-LLaMA and kept frozen.
6:   Hash Model:  $H(\cdot; \mu)$ , randomly initialized and trainable.
7: for each training batch  $D_i \subset S$  do
8:   Feature extraction:  $\mathbf{z}_i = F(D_i; \theta)$ .
9:   Compute hash codes:  $\{\mathbf{h}_i\}_{i=1}^c, \mathbf{o} = H(\mathbf{z}_i; \mu)$ .
10:  Compute  $\mathcal{L}_{\text{InfoNCE}}$ .
11:  Update  $\mu$  while keeping  $\theta$  frozen.
12: end for
13: /*———— Stage 2: Quantization and Adaptation ———*/
14: Quantize teacher model Video-HLM:
15:   Quantize  $F(\cdot; \theta)$  to NF4 format:  $\tilde{F}(\cdot; \theta^{\text{NF4}})$ .
16:   Quantize  $H(\cdot; \mu)$  to NF2 format:  $\hat{H}(\cdot; \mu^{\text{NF2}})$ .
17: Insert adaptation module  $\omega$  into  $\hat{H}$ , forming adapted hash model  $\hat{H}(\cdot; \theta^{\text{NF2}}, \omega)$ .
18: Then we get student model:  $\text{QuFiH}(\cdot) = \hat{H}(\tilde{F}(\cdot; \theta^{\text{NF4}}); \mu^{\text{NF2}}, \omega)$ .
19: /*———— Stage 3: Fine-Tuning the Quantized Model ———*/
20: /*—— Strategy 1: Joint Optimization ——*/
21: for each training batch  $D_i \subset S$  do
22:   Extract teacher features:  $\mathbf{z}_i^T = F(D_i; \theta)$ .
23:   Compute teacher hash codes:  $\{\mathbf{h}_i^T\}_{i=1}^c, \mathbf{o}^T = H(\mathbf{z}_i^T; \mu)$ .
24:   Extract student features:  $\mathbf{z}_i^S = \tilde{F}(D_i; \theta^{\text{NF4}})$ .
25:   Compute student hash codes:  $\{\mathbf{h}_i^S\}_{i=1}^c, \mathbf{o}^S = \hat{H}(\mathbf{z}_i^S; \mu^{\text{NF2}}, \omega)$ .
26:   Compute distillation loss:  $\mathcal{L}_{\text{distill}} = \sum_{i=1}^c D(\mathbf{h}_i^T, \mathbf{h}_i^S) + 2 \cdot D(\mathbf{o}^T, \mathbf{o}^S)$ 
27:   Compute InfoNCE loss:  $\mathcal{L}_{\text{InfoNCE}}$ .
28:   Compute total loss:  $\mathcal{L}_{\text{total}} = \alpha \mathcal{L}_{\text{distill}} + (1 - \alpha) \mathcal{L}_{\text{InfoNCE}}$ .
29:   Update  $\omega$  via backpropagation.
30: end for
31: /*—— Strategy 2: Two-Stage Training ——*/
32: for each training batch  $D_i \subset S$  do
33:   Extract teacher features:  $\mathbf{z}_i^T = F(D_i; \theta)$ .
34:   Compute teacher hash codes:  $\{\mathbf{h}_i^T\}_{i=1}^c, \mathbf{o}^T = H(\mathbf{z}_i^T; \mu)$ .
35:   Extract student features:  $\mathbf{z}_i^S = \tilde{F}(D_i; \theta^{\text{NF4}})$ .
36:   Compute student hash codes:  $\{\mathbf{h}_i^S\}_{i=1}^c, \mathbf{o}^S = \hat{H}(\mathbf{z}_i^S; \mu^{\text{NF2}}, \omega)$ .
37:   Compute distillation loss:  $\mathcal{L}_{\text{distill}} = \sum_{i=1}^c D(\mathbf{h}_i^T, \mathbf{h}_i^S) + 2 \cdot D(\mathbf{o}^T, \mathbf{o}^S)$ 
38:   Update  $\omega$  via backpropagation.
39: end for
40: for each training batch  $D_i \subset S$  do
41:   Extract student features:  $\mathbf{z}_i^S = \tilde{F}(D_i; \theta^{\text{NF4}})$ .
42:   Compute student hash codes:  $\{\mathbf{h}_i^S\}_{i=1}^c, \mathbf{o}^S = \hat{H}(\mathbf{z}_i^S; \mu^{\text{NF2}}, \omega)$ .
43:   Compute InfoNCE loss:  $\mathcal{L}_{\text{InfoNCE}}$ .
44:   Update  $\omega$  via backpropagation.
45: end for
46: return QuFiH.

```

Table 1

Dataset statistics.

Items\Datasets	ActivityNet	FCVID
#Training	10,023	45,508
#Validation	2,512	22,754
#Testing	2,413	22,754
#Categories	200	239
#Retrieval database	10,023	45,508
#Queries	2,413	22,754

The overall statistics of these datasets are summarized in Table 1.

4.2. Baselines and metrics

Baselines. We compared our QuFiH method with several SOTA video hashing approaches, including SSTH⁴ (Zhang et al., 2016), BTH⁵ (Li et al., 2021), DKPH⁶ (Li et al., 2022), MCMSH⁷ (Hao et al., 2022), ConMH⁸ (Wang, Wang et al., 2023), S5VH⁹ (Wang et al., 2025), SRH¹⁰ (Gu et al., 2016), AVH¹¹ (Wang et al., 2021), DSVH¹² (Chen, Hu et al., 2021), and AVHash¹³ (Zhou, Sun, Liu, Chen, & Zhang, 2024). The first six competitors are unsupervised methods, while the last four are supervised ones.

Metrics. We evaluated video retrieval performance using three widely-adopted metrics: mean Average Precision at the top- K retrieved results, i.e., $\text{mAP}@K$, (usually $\text{mAP}@100$), Precision at the top- K retrieved results (i.e., $\text{P}@K$), and Recall at the top- K retrieved results (i.e., $\text{R}@K$) (Pu et al., 2025; Sun, Dai et al., 2024; Sun, Ren, Hu, Peng and Wang, 2024; Wang et al., 2021). Additionally, we plotted P-R (Precision vs. Recall) curves to make comprehensive comparisons between the video retrieval behaviors of different methods.

4.3. Implementation details

Image Processing. We uniformly sample 25 frames from each video in the ActivityNet and FCVID datasets as visual input. These frames are then processed by the visual branch of Video-LLaMA. This branch utilizes the pre-trained visual component of BLIP-2 (Li, Li, Savarese et al., 2023) as a frozen visual encoder, with its core being the ViT-G/14 model based on EVA-CLIP (Fang et al., 2023) and a pre-trained Q-Former.

Audio Processing. For audio processing, we divide each audio track in the ActivityNet and FCVID datasets into 25 segments, corresponding to the video frames. These audio segments are processed by the audio branch of Video-LLaMA. This branch first employs the pre-trained ImageBind (Girdhar et al., 2023) as the audio encoder. Then, an Audio Q-Former is used to add learnable positional embeddings to the audio segments, injecting temporal information. The Audio Q-Former generates fixed-length audio features by computing interactions between position-encoded audio segments. Its architecture is consistent with that of the Q-Former. Finally, a linear module maps the audio features into the embedding space of the language model.

Network Architecture. Our network architecture consists of three core components: a modality interaction encoder, a multimodal encoder, and a hash layer. Both the modality interaction encoder and the multimodal encoder adopt a Transformer-based architecture, configured with a single block, 12 multi-head attention mechanisms, and a 768-dimensional hidden layer. To simplify the model and improve efficiency, we limit the number of blocks in each component to one. The hash layer maps the 768-dimensional hidden vectors to hash codes of a specified length, generating the final video hash representation. This design ensures the model's expressive power while significantly reducing computational complexity.

Model Training. The model training consists of two stages:

1. Full-Precision Pretraining. In this stage, we use the Adam optimizer (Kingma & Ba, 2015) for gradient descent, with a learning rate of 10^{-4} and a batch size of 128. During training, four negative samples ($n_o = 4$) are used, and the temperature parameter is set to $\tau = 0.1$. The lower foundation model is frozen, and only the upper hash model is trained. This ensures that the model learns effective hash representations while preserving the generalization ability of the pretrained feature encoders.

2. Quantized Model Fine-Tuning. The fine-tuning stage also employs the Adam optimizer with a learning rate of 10^{-4} and a batch size of 128. This stage consists of two tasks:

(1) Knowledge Distillation. The student model (quantized model) is aligned with the teacher model (full-precision model) at the outputs of the modality interaction encoder, multimodal encoder, and hash layer to achieve knowledge distillation, thereby improving the performance of the quantized model.

(2) Downstream Task Optimization. The InfoNCE loss function is used with four negative samples ($n_o = 4$) and a temperature parameter $\tau = 0.1$ to further optimize the model.

During this stage, only the newly added LoRA or Propulsion modules are trained, significantly reducing the number of parameters and computational cost while maintaining model performance. By following this two-stage training strategy, the model achieves efficient and high-performance video hash representation learning, leveraging full-precision pretraining followed by quantization-aware fine-tuning.

⁴ <https://github.com/hanwangzhang/BLSTM>.

⁵ <https://github.com/Lily1994/BTH>.

⁶ <https://github.com/IMCCretrieval/DKPH>.

⁷ <https://github.com/haoyanbin918/MCMSH>.

⁸ <https://github.com/huangmozhi9527/ConMH>.

⁹ <https://github.com/gimpong/AAAI25-S5VH>.

¹⁰ Its codes are reproduced base on <https://pytorch.org/docs/stable/generated/torch.nn.LSTM.html>.

¹¹ Its codes are reproduced base on <https://pytorch.org/docs/stable/generated/torch.nn.LSTM.html>.

¹² <https://github.com/kenshohara/3D-ResNets-PyTorch>.

¹³ <https://github.com/iFamily/AVHash>.

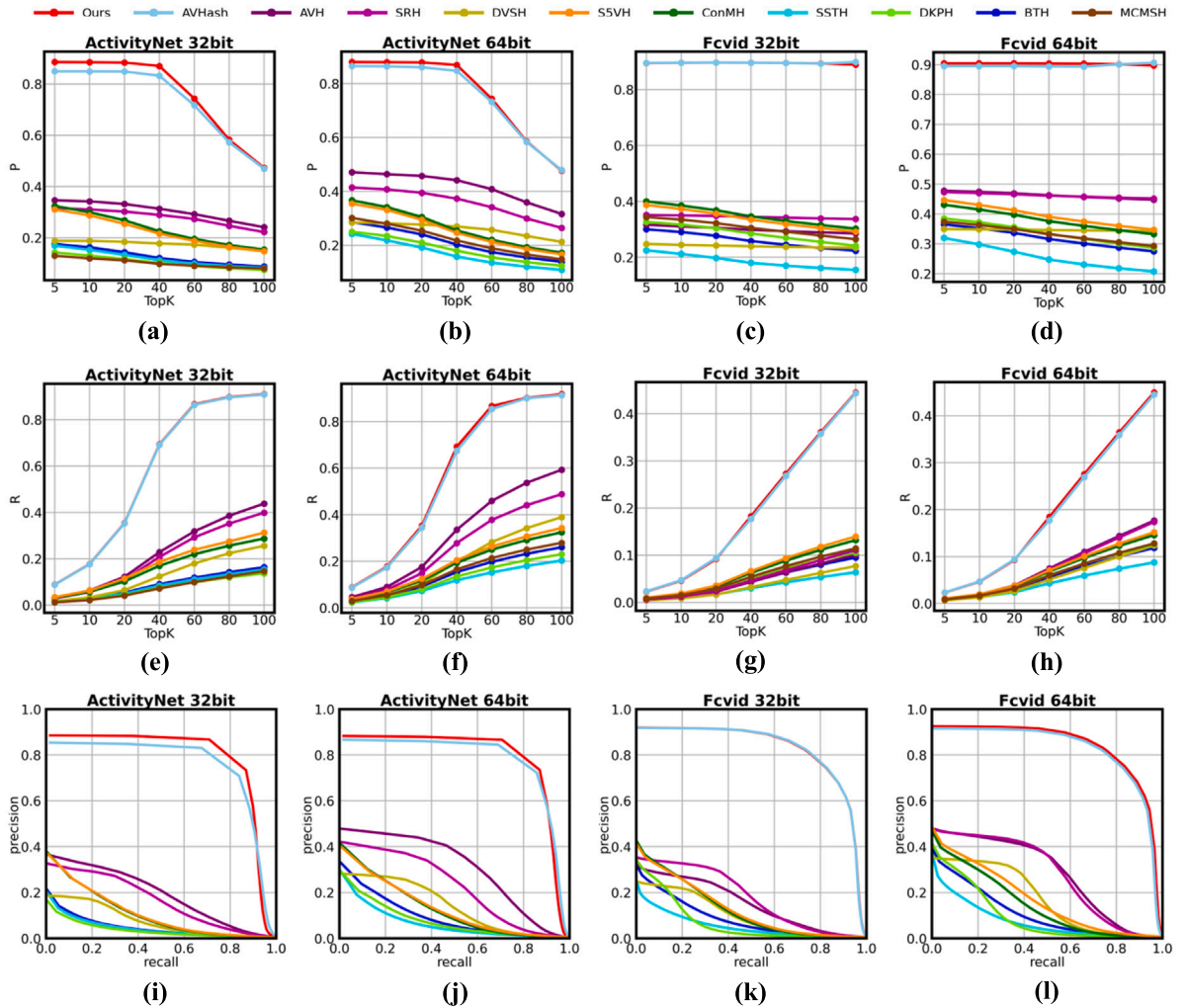


Fig. 3. Performance comparisons w.r.t. $P@K$, $R@K$, and P-R curve on ActivityNet and FCVID datasets (32 bits and 64 bits). Note that the mAP@100 results are reported in Table 2.

4.4. Results

Table 2 presents the mAP@100 results of various video hashing methods on the ActivityNet and FCVID datasets. We comprehensively evaluate the performance of our proposed QuFiH method, which combines hybrid low-bit quantization with parameter-efficient fine-tuning, against existing approaches for video retrieval tasks.

Among existing methods, AVHash demonstrates superior performance across both datasets and various bit settings, significantly outperforming traditional approaches. For instance, at 64 bits on ActivityNet, AVHash achieves an mAP of 0.8676, substantially exceeding the second-best method SRH (0.4266). Similarly, on FCVID at 64 bits, AVHash attains an mAP of 0.8913 compared to AVH's 0.4818. These results indicate AVHash's effectiveness in capturing semantic video information and generating high-quality hash codes.

Our proposed QuFiH method achieves remarkable performance improvements on both datasets, surpassing existing approaches including AVHash. Specifically, on ActivityNet at 64 bits, QuFiH (hybrid quantization + LoRA, rank = 4) enhances the mAP to 0.8876, representing a 0.02 improvement over AVHash. On FCVID at 64 bits, QuFiH increases the mAP to 0.9111, outperforming AVHash by 0.0198. The experimental results demonstrate that QuFiH, through its integration of hybrid low-bit quantization strategy and parameter-efficient fine-tuning techniques, significantly enhances video retrieval accuracy while reducing computational and storage overhead.

In contrast, traditional hashing methods demonstrate considerably lower mAP scores, typically below 0.5. For instance, at the 32-bit configuration, AVH's mAP (0.3576) is approximately 2.48 times lower than that of hybrid quantization + LoRA (0.8861),

Table 2mAP@100 results of different methods on the **ActivityNet** and **FCVID** datasets with 32-bit and 64-bit hash codes.

Methods	ActivityNet ($K = 100$)		FCVID ($K = 100$)	
	32 bits	64 bits	32 bits	64 bits
MCMSH (Hao et al., 2022)	0.1488	0.2849	0.3442	0.3723
BTH (Li et al., 2021)	0.1845	0.2721	0.2987	0.3593
DKPH (Li et al., 2022)	0.1528	0.2283	0.3256	0.3731
SSTH (Zhang et al., 2016)	0.1727	0.2395	0.2275	0.3037
ConMH (Wang, Wang et al., 2023)	0.2974	0.3312	0.3899	0.4232
S5VH (Wang et al., 2025)	0.3127	0.3465	0.4057	0.4437
DSVH (Chen, Hu et al., 2021)	0.2056	0.2979	0.2616	0.3683
SRH (Gu et al., 2016)	0.3205	0.4266	0.3699	0.4783
AVH (Wang et al., 2021)	0.3576	0.4722	0.3318	0.4818
AVHash (Zhou et al., 2024)	0.8530	0.8676	0.8908	0.8913
Video-HLM	0.8421	0.8678	0.8972	0.9061
QuFiH+L-LoRA (rank = 4)	0.8861	0.8876	0.9046	0.9111
QuFiH+B-LoRA (rank = 4)	0.8778	0.8771	0.8949	0.9007
QuFiH+L-Propulsion (degree = 5)	0.8799	0.8782	0.9024	0.9043
QuFiH+B-Propulsion (degree = 5)	0.8622	0.8833	0.8852	0.8942

Table 3

Ablation results of parameter-efficient fine-tuning methods (LoRA and Propulsion) on ActivityNet. L-* denotes Linear-Level, B-* denotes Block-Level, "r" is LoRA rank, and "d" is Propulsion degree.

Methods/mAP/datasets		LoRA						Propulsion					
		32 bits			64 bits			32 bits			64 bits		
		r = 4	r = 8	r = 16	r = 4	r = 8	r = 16	d = 5	d = 15	d = 25	d = 5	d = 15	d = 25
L-*	Weight-Sum	0.8806	0.8811	0.8818	0.8875	0.8897	0.8879	0.8760	0.8756	0.8784	0.8782	0.8833	0.8820
	Two-Stage	0.8861	0.8857	0.8888	0.8876	0.8878	0.8875	0.8799	0.8840	0.8827	0.8833	0.8861	0.8852
B-*	Weight-Sum	0.8617	0.8681	0.8720	0.8712	0.8770	0.8805	0.8461	0.8472	0.8472	0.8580	0.8580	0.8578
	Two-Stage	0.8778	0.8788	0.8839	0.8771	0.8774	0.8811	0.8622	0.8684	0.8722	0.8649	0.8692	0.8722
Only quantization		0.6625			0.7650			0.6625			0.7650		
Video-HLM		0.8421			0.8678			0.8421			0.8678		

while SRH (0.3205) underperforms hybrid quantization + Propulsion (0.8799) by approximately 2.74 times. These findings highlight the significant advantages of large-model-based quantization methods in video retrieval tasks.

Furthermore, LoRA and Propulsion methods exhibit varying performance across different structural implementations. Linear-level LoRA outperforms block-level LoRA, whereas block-level Propulsion slightly exceeds linear-level implementation at 64 bits (0.8833 vs. 0.8782), suggesting differential adaptability of various strategies to different architectural configurations.

Notably, traditional methods display substantial performance disparities between 32-bit and 64-bit configurations (e.g., AVH improves from 0.3576 to 0.4722), while hybrid quantization + LoRA and Propulsion methods maintain relatively consistent performance across different bit widths. For example, LoRA's mAP differs by merely 0.0015 between 32-bit and 64-bit settings, demonstrating these methods' ability to maintain stable high performance across various code lengths. This consistency likely stems from efficient feature learning strategies that preserve robust retrieval capabilities even with shorter hash codes.

Additionally, we collected P@K, R@K, and P-R metrics for all methods and plotted the corresponding curves (as shown in Fig. 3). The results demonstrate that QuFiH outperforms existing video hashing methods on both P@K and R@K metrics, further validating its significant advantages in retrieval precision and recall rates, while exhibiting clear superiority on the P-R curve.

In conclusion, our experimental results comprehensively demonstrate that QuFiH excels in video hashing retrieval tasks, not only significantly outperforming traditional hashing methods but also surpassing the full-precision trained Video-HLM large model. These findings establish QuFiH as a superior solution for efficient video retrieval applications.

4.5. Ablation studies

Table 3 presents the experimental results of fine-tuning video hashing models using different parameter-efficient fine-tuning methods (LoRA and Propulsion) and different hashcode sizes (32-bit and 64-bit) on the ActivityNet dataset. The table compares two fine-tuning strategies: Weight-Sum and Two-Stage, and provides the mAP performance metrics under various hyperparameter settings. Additionally, the results of using only quantization and the Video-HLM baseline method are listed as references for performance comparisons.

For LoRA adaptation, increasing the rank value consistently improves performance across both linear-level and block-level structures. In the Weight-Sum approach with 32-bit settings, mAP increases from 0.8806 (Rank = 4) to 0.8818 (Rank = 16). The linear-level structure shows more pronounced improvements, with the Two-Stage approach at 32 bits demonstrating an mAP increase from 0.8778 (Rank = 4) to 0.8839 (Rank = 16), indicating better utilization of higher-dimensional feature representations.

Table 4

Ablation results of parameter-efficient fine-tuning methods (LoRA and Propulsion) on FCVID. L-* denotes Linear-Level, B-* denotes Block-Level, "r" is LoRA rank, and "d" is Propulsion degree.

methods/mAP/datasets		LoRA						Propulsion					
		32 bits			64 bits			32 bits			64 bits		
		r = 4	r = 8	r = 16	r = 4	r = 8	r = 16	d = 5	d = 15	d = 25	d = 5	d = 15	d = 25
L-*	Weight-Sum	0.9066	0.9070	0.9074	0.9080	0.9091	0.9088	0.9027	0.9075	0.9087	0.9039	0.9065	0.9069
	Two-Stage	0.9046	0.9080	0.9083	0.9111	0.9127	0.9151	0.9024	0.9081	0.9080	0.9043	0.9071	0.9083
B-*	Weight-Sum	0.8990	0.9018	0.9042	0.9005	0.9027	0.9050	0.8860	0.8864	0.8867	0.8930	0.8937	0.8936
	Two-Stage	0.8949	0.9016	0.9042	0.9007	0.9061	0.9115	0.8852	0.8866	0.8869	0.8942	0.8951	0.8952
Only quantization			0.7439			0.7898			0.7439			0.7898	
Video-HLM			0.8972			0.9061			0.8972			0.9061	

The Two-Stage approach consistently outperforms Weight-Sum across all configurations. At 32 bits with Rank = 16, Two-Stage achieves an mAP of 0.8888, surpassing Weight-Sum's 0.8818. This advantage becomes more pronounced in block-level structures, particularly at higher rank values, suggesting more effective LoRA adaptation strategy in the Two-Stage approach.

For Propulsion adaptation, linear-level structures show modest improvements with increasing Degree values. At 32 bits, Weight-Sum's mAP improves from 0.8760 to 0.8784, while Two-Stage increases from 0.8799 to 0.8827. Block-level structures demonstrate more significant sensitivity to Degree parameter adjustments, with Two-Stage at 32 bits showing a notable improvement from 0.8622 to 0.8722.

The Two-Stage approach consistently demonstrates superior performance across all Propulsion configurations. At 32 bits with Degree = 25, Two-Stage achieves an mAP of 0.8827 in linear-level structure, exceeding Weight-Sum's 0.8784. This advantage is particularly evident in block-level structures, where Two-Stage achieves an mAP of 0.8722, significantly outperforming Weight-Sum's 0.8472. These results suggest that the Two-Stage approach implements more effective adaptation strategy for both LoRA and Propulsion techniques.

Table 4 presents the mAP@100 results of LoRA and Propulsion methods on the FCVID dataset, revealing patterns consistent with those observed on ActivityNet.

For LoRA adaptation, increasing rank values generally leads to improved performance across all configurations. The Two-Stage approach consistently outperforms Weight-Sum, suggesting more effective model design and training strategy. Notably, linear-level structures achieve marginally higher mAP compared to block-level structures, indicating better capability in capturing relationships within the data.

Similarly, for Propulsion adaptation, higher Degree values correlate with improved mAP@100 scores. The Two-Stage approach maintains superior performance over Weight-Sum across different configurations. The linear-level structures consistently demonstrate slightly better performance than block-level structures, further supporting their effectiveness in feature representation.

By comparing the experimental results of LoRA and Propulsion methods on the ActivityNet and FCVID datasets, we can draw the following conclusions:

1. The Two-Stage approach consistently outperforms Weight-Sum across both methods and structural levels. For LoRA at 32 bits (Rank = 16), Two-Stage achieves an mAP of 0.8888 compared to Weight-Sum's 0.8818 at the linear level. Similarly, for Propulsion (Degree = 25), Two-Stage reaches 0.8827 versus Weight-Sum's 0.8784.

2. Linear-level structures demonstrate superior performance over block-level structures across both methods. With Two-Stage at 32 bits, LoRA achieves 0.8888 (linear-level) versus 0.8839 (block-level), while Propulsion shows 0.8827 (linear-level) versus 0.8722 (block-level).

3. Although the mAP results of the block-level method are slightly lower than those of the linear-level, it significantly outperforms the linear-level in terms of the number of parameters. The performance of the block-level method is close to that of the linear-level, within an acceptable range. Under certain specific configurations, the performance difference between the block-level method and the linear-level is very small and can be almost neglected. This indicates that the block-level method can maintain performance comparable to the linear-level while significantly reducing the number of parameters, demonstrating its superior parameter efficiency and practical value. Therefore, the block-level method provides a highly attractive option when balancing model performance and computational resources, especially in resource-constrained scenarios, where it can significantly reduce model complexity and computational overhead while ensuring performance.

LoRA vs. Propulsion. When comparing the two parameter-efficient fine-tuning methods directly, LoRA generally demonstrates a performance advantage over Propulsion in our experiments. Across both datasets, LoRA tends to reach a higher mAP ceiling. Using the optimal Two-Stage, linear-level setup on ActivityNet (32-bit), LoRA's peak mAP is 0.8888 ($r = 16$), whereas Propulsion's peak is 0.8827 ($d = 25$). This pattern is consistent across different settings. For example, on FCVID with the Two-Stage, block-level approach (64-bit), LoRA ($r = 16$) achieves an mAP of 0.9115, which is considerably higher than Propulsion's 0.8952 ($d = 25$).

While both methods show performance gains as their respective hyperparameters (rank for LoRA, degree for Propulsion) increase, LoRA appears to be more effective at leveraging these additional parameters to achieve superior retrieval accuracy in the context of our video hashing framework.

Linear-level vs. Block-level. In terms of raw mAP scores, linear-level structures consistently achieve slightly better results than block-level structures. For example, on ActivityNet using the Two-Stage LoRA method (32-bit, $r = 16$), the linear-level mAP is

Table 5

The mAP@100 values of Video-HLM using different input modalities (visual-only, audio-only, or audio-visual) on ActivityNet and FCVID datasets with 32 and 64 bits.

Method/mAP/Modality	ActivityNet		FCVID	
	32 bits	64 bits	32 bits	64 bits
Audio	0.2020	0.2208	0.3989	0.4168
Visual	0.8163	0.8507	0.8681	0.8819
Visual+audio	0.8421	0.8678	0.8972	0.9061

0.8888, while the block-level is 0.8839. This trend holds true for Propulsion and on the FCVID dataset as well. The key advantage of the block-level method is its significantly lower parameter count. Although its mAP results are marginally lower, the performance is often very close to that of the linear-level, with the gap being negligible in some cases. This demonstrates that the block-level structure can maintain highly competitive performance while drastically reducing model complexity and computational overhead.

Therefore, the block-level method presents a highly attractive option for resource-constrained scenarios, offering a superior balance between model performance and computational cost.

Two-Stage vs. Weight-Sum. The Two-Stage training strategy consistently and significantly outperforms the Weight-Sum approach across nearly all configurations, including both LoRA and Propulsion methods on both datasets. This suggests that the Two-Stage approach provides a more effective adaptation strategy for the hashing task.

On ActivityNet, this advantage is prominent. For instance, with 32-bit LoRA ($r = 16$) at the linear level, the Two-Stage approach achieves an mAP of 0.8888, markedly surpassing Weight-Sum's 0.8818. The gap is even larger with Propulsion at the block level ($d = 25$), where Two-Stage (0.8722) is substantially better than Weight-Sum (0.8472). On FCVID, this pattern is replicated. The highest overall mAP of 0.9151 is achieved by the Two-Stage method (LoRA, 64-bit, $r = 16$), clearly outperforming the corresponding Weight-Sum score of 0.9088.

This consistent outperformance indicates that separating the fine-tuning process into two stages allows the model to better adapt its features for video retrieval.

Single-Signal vs. Multi-Signal. To elucidate the individual and joint contributions of audio and visual modalities to the performance of Video-HLM in video retrieval tasks, we conducted a series of ablation experiments. Specifically, we compared the full bimodal configuration (“audio+visual”) against two unimodal variants that utilize only the “visual” or “audio” modality.

As shown in Table 5, the bimodal model consistently outperforms both unimodal counterparts, with the visual-only variant achieving higher performance than the audio-only one. Notably, while the visual modality alone provides a strong baseline in terms of retrieval accuracy (mAP@100), the inclusion of audio signals yields a clear performance gain.

These results highlight the complementary nature of multimodal information and underscore the effectiveness of modality fusion in enhancing retrieval quality within the Video-HLM framework.

4.6. Performance of different hybrid precision quantization

Analysis of quantization experiments on ActivityNet and FCVID datasets (Table 6) reveals several key findings:

1. Quantization can alleviate overfitting: After quantizing and fine-tuning the full-precision model, it not only improves the model's performance but also further alleviates the overfitting problem. Through quantization, the model can better adapt to the characteristics of specific tasks and datasets, learning more robust and generalized feature representations, which acts as a form of regularization.

2. The combination of quantization and parameter-efficient fine-tuning provides greater flexibility: Performing parameter-efficient fine-tuning on the basis of a quantized model, even when using lower quantization precision, can compensate for the performance loss caused by quantization through fine-tuning, achieving comparable or even better results than high-precision quantization. This provides more choices for model compression and acceleration in practical applications.

3. The hybrid low-bit quantization strategy has advantages: The hybrid low-bit quantization strategy, which employs different quantization precisions for different parts of the model, maintains high performance while reducing the model size. Combining this hybrid low-bit quantization strategy with parameter-efficient fine-tuning can further exploit the synergistic effects of both, achieving better performance.

4. High-precision representations of large model bases are not necessary for video hashing tasks: By comparing the experimental results, it can be found that 4-bit quantization of the large model base is sufficient to support the video hashing encoding capability, and high-precision representations are not necessary.

4.7. Distillation parameter analysis

The key parameters in our experiments include the learning rate, batch size, and the weight ratio between the distillation task and the downstream task (distillation coefficient α).

According to the experience of related works, such as AVHash (Zhou et al., 2024), when the learning rate is set to $1e-4$ and the batch size is 128, the model typically achieves optimal performance. Therefore, in our experiments, we directly adopted these parameter settings and did not conduct a separate analysis of the learning rate. Instead, we focused on investigating the impact of

Table 6

Impact of different quantization strategies on video hashing performance (mAP@100).

Foundation model+Hash model	ActivityNet		FCVID	
	32 bits	64 bits	32 bits	64 bits
4 bit + 2 bit (QuFiH)	0.8875	0.8888	0.9007	0.9058
4 bit + 4 bit	0.8908	0.8913	0.9069	0.9083
4 bit + float32	0.8919	0.8930	0.9074	0.9089
float32 + 2 bit	0.8877	0.8888	0.8969	0.9074
float32 + 4 bit	0.8918	0.8949	0.8997	0.9108
float32 + float32 (Video-HLM)	0.8421	0.8678	0.8972	0.9061

Table 7Impact of different distillation coefficients α on model performance (mAP@100).

Dataset/mAP@100/ α		α value							
		0	0.1	0.2	0.4	0.5	0.6	0.8	1.0
ActivityNet	L-Propulsion	0.8769	0.8833	0.8820	0.8783	0.8770	0.8743	0.8721	0.8568
	L-LoRA	0.8719	0.8875	0.8825	0.8751	0.8793	0.8789	0.8759	0.8497
FCVID	L-Propulsion	0.8972	0.9043	0.9010	0.9006	0.9020	0.9021	0.9017	0.9015
	L-LoRA	0.8927	0.9017	0.8987	0.8979	0.8970	0.9008	0.9011	0.9010

Table 8

Parameter storage space (in bytes) before and after quantization.

Parameter storage (Bytes)	Before quantization	After quantization	Ratio
Foundation model	8,022,237,188	1,194,712,708	6.71
Hash model	292,557,056	20,794,112	14.06
Video-HLM	8,314,794,244	1,215,506,820	6.84

the distillation coefficient α on the model's performance, aiming to find the optimal balance between the distillation task and the downstream task, further enhancing the model's performance on video representation tasks.

Table 7 presents the experimental results of the Weight-Sum approach under different distillation ratios, where α represents the weight of the distillation task and $1-\alpha$ represents the weight of the downstream task.

The results demonstrate that model performance reaches its peak when $\alpha = 0.1$, with mAP values significantly higher than other configurations. This finding indicates that moderate knowledge distillation effectively enhances model performance by extracting valuable information from the teacher model, thereby improving the student model's generalization capability.

Based on these findings, subsequent experiments adopt $\alpha = 0.1$ as the optimal distillation weight configuration for the Weight-Sum approach. This setting ensures an optimal balance between knowledge distillation and downstream task optimization. The results not only validate the effectiveness of the distillation strategy but also provide valuable insights for weight allocation in multitask learning scenarios.

4.8. Quantization efficiency analysis

Quantization brings notable benefits in terms of both memory usage and inference performance. We present a detailed analysis of these two aspects in the following subsections.

4.8.1. Memory efficiency analysis

Table 8 demonstrates the comparison of parameter storage space before and after quantization for foundation model and hash model. Analysis of the data yields the following conclusions:

1. **Quantization Effect on Foundation Model.** After 4-bit quantization, the parameter storage space of foundation model decreased from 8,022,237,188 bytes to 1,194,712,708 bytes, achieving a compression ratio of 6.71. This indicates that 4-bit quantization can reduce the storage space of foundation model by approximately 80%, significantly lowering the model's storage overhead.

2. **Quantization Effect on Hash Model.** After 2-bit quantization, the parameter storage space of hash model decreased from 292,557,056 bytes (approximately 292.56 MB) to 20,794,112 bytes (approximately 20.79 MB), achieving a high compression ratio of 14.06. This means 2-bit quantization reduces the hash model's storage space by about 93%. Compared to the 4-bit quantization of foundation model, the 2-bit quantization of hash model achieved higher compression efficiency.

3. **Overall Quantization Effect on the Video Hash Model.** After quantization, the total parameter storage space decreased from 8,314,794,244 bytes to 1,215,506,820 bytes, with an overall compression ratio of 6.84. This result demonstrates that the hybrid low-bit quantization strategy (4-bit for foundation model and 2-bit for hash model) effectively maintains model performance while significantly reducing storage costs.

4. **Advantages of Low-Bit Quantization for Hash Model.** Although the hash model has relatively fewer parameters, its compression ratio is significantly higher than the foundation model due to the more aggressive 2-bit quantization. This phenomenon indicates

Table 9

Comparison of LoRA and Propulsion in terms of parameter count under different configurations.

Parameter number	LoRA			Propulsion		
	r = 4	r = 8	r = 16	d = 5	d = 15	d = 25
L-*	556,288	1,112,576	2,225,152	119,424	119,424	119,424
B-*	122,112	244,224	488,448	13,632	13,632	13,632

Table 10

Different video hashing methods' inference time on ActivityNet and FCVID datasets with the code length of 64 bits. Note that "m" denotes minutes, "s" represents seconds, and "ms" represents miniseconds. (tested on NVIDIA V100 GPU 32 GB).

Method/Time/Dataset	ActivityNet		FCVID	
	Inference time		Inference time	
	Total time for 12.5k videos	Average time for one video	Total time for 68k videos	Average time for one video
MCMSH (Hao et al., 2022)	13 s	1.04 ms	1 m 1 s	0.89 ms
BTH (Li et al., 2021)	15 s	1.20 ms	1 m 13 s	1.07 ms
DKPH (Li et al., 2022)	12 s	0.96 ms	59 s	0.86 ms
SSTH (Zhang et al., 2016)	29 s	2.31 ms	2 m 51 s	2.50 ms
ConMH (Wang, Wang et al., 2023)	10 s	0.79 ms	49 s	0.72 ms
SSVH (Wang et al., 2025)	7 s	0.55 ms	36 s	0.53 ms
DSVH (Chen, Hu et al., 2021)	1 m 11 s	5.66 ms	6 m 19 s	5.55 ms
SRH (Gu et al., 2016)	9 s	0.72 ms	47 s	0.69 ms
AVH (Wang et al., 2021)	8 s	0.64 ms	41 s	0.60 ms
AVHash (Zhou et al., 2024)	6 s	0.47 ms	33 s	0.48 ms
Video-HLM	14 m 55 s	71.40 ms	81 m 16 s	71.43 ms
QuFiH	12 m 56 s	61.90 ms	70 m 32 s	62.00 ms

that applying low-bit quantization (such as 2-bit) to layers with fewer parameters can further reduce storage overhead without significant accuracy loss, providing crucial support for model deployment in resource-constrained scenarios.

In conclusion, the application of hybrid low-bit quantization strategy on feature encoding and hash layer not only significantly reduces the model's storage requirements, but also achieves a balance between performance and efficiency through reasonable bit allocation, providing a viable solution for deploying efficient video hash models.

Besides, Table 9 presents a comparison of the parameter sizes of two parameter-efficient fine-tuning methods: LoRA and Propulsion, under different configurations. Based on the data in the table, the following conclusions can be drawn:

1. LoRA's parameter size increases significantly with rank, while Propulsion remains stable. LoRA fine-tunes the model by introducing trainable low-rank matrices, resulting in a linear increase in parameter size with rank. For example, when the rank increases from 4 to 16, the number of parameters grows from 556,288 to 2,225,152, a fourfold increase. B-LoRA exhibits a similar trend. In contrast, Propulsion maintains a stable parameter size regardless of rank changes.

2. Propulsion is more parameter-efficient than LoRA. The number of additional parameters introduced by Propulsion is significantly smaller than LoRA. For instance, when rank $r = 16$, LoRA introduces 2,225,152 parameters at the linear level and 488,448 parameters at the block level. Meanwhile, Propulsion, even at $d = 15$, introduces only 119,424 parameters at the linear level and 13,632 parameters at the block level. This demonstrates that Propulsion has a clear advantage in parameter efficiency.

3. LoRA requires a trade-off between performance and efficiency, while Propulsion is better suited for resource-constrained scenarios. In LoRA, a higher rank improves the model's expressive power but also increases parameter size, leading to higher computational and storage overhead. Therefore, LoRA is more suitable for scenarios requiring high performance and ample resources. By contrast, Propulsion enables efficient fine-tuning with fewer parameters, making it a better choice for resource-constrained environments.

In conclusion, Propulsion demonstrates superior parameter efficiency compared to LoRA, offering a better fine-tuning solution for resource-constrained scenarios, while LoRA remains suitable for high-performance requirements.

4.8.2. Inference efficiency analysis

While the above discussion primarily focuses on the storage efficiency of our quantized model, we now turn to analyzing its inference-time efficiency.

In this work, our QuFiH framework adopts a hybrid 2/4-bit quantization scheme to compress model parameters. In this scheme, 2-bit values serve as a fundamental quantization unit. However, due to the lack of native 2-bit data type support in Python, we utilize the int8 data type to store the 2-bit quantized values. Specifically, each int8 can accommodate four 2-bit values. Although this approach reduces memory usage to a certain extent, it introduces non-negligible computational overhead during inference. This is primarily due to the bitwise operations required for dequantization, which are inefficient in software environments lacking native 2-bit computation support.

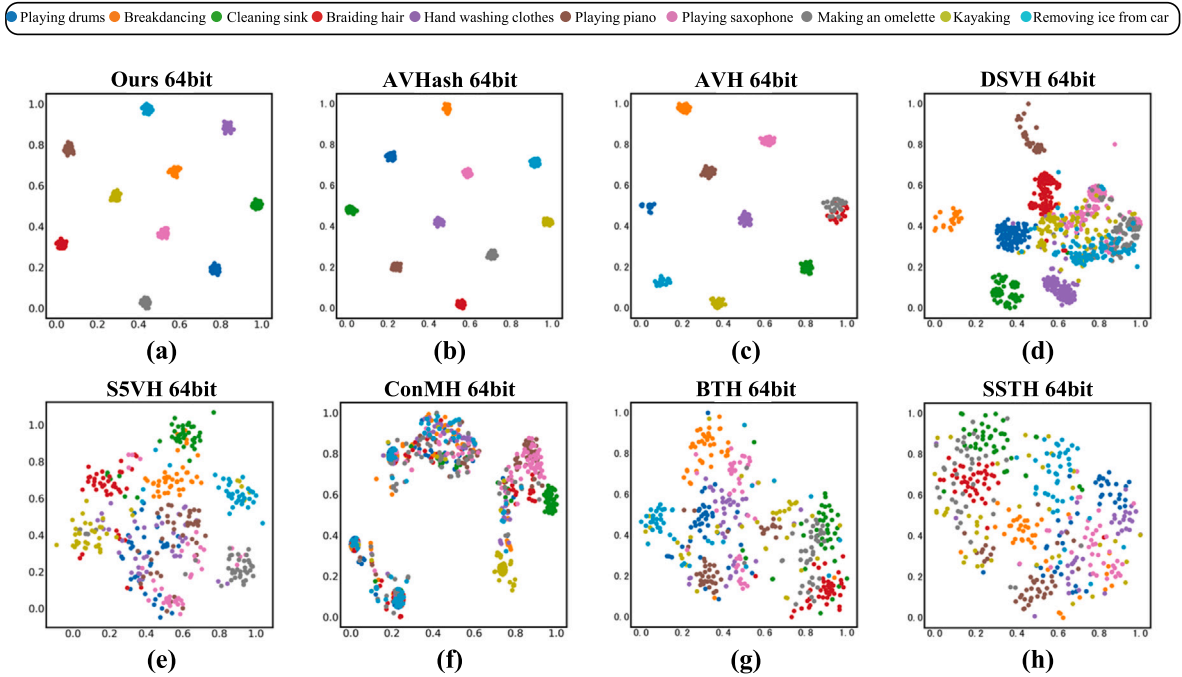


Fig. 4. t-SNE visualizations of QuFiH and 7 most competitive methods on ActivityNet dataset.

Given the current absence of mature frameworks optimized for 2-bit inference, we refer to the implementation details provided in FLUX (Yang et al., 2024), which offers a practical 2-bit quantization computation framework. Based on this, we benchmark the inference latency of QuFiH across various test video datasets. For comparison, we also report the inference times of Video-HLM and other representative baseline methods in Table 10. All experiments are conducted using an NVIDIA V100 GPU with 32 GB memory to satisfy the requirements of the large-scale Video-HLM model.

As shown in the table, conventional non-large-model methods—such as MCMSH (Hao et al., 2022), ConMH (Wang, Wang et al., 2023), S5VH (Wang et al., 2025), AVH (Wang et al., 2021), and AVHash (Zhou et al., 2024)—typically exhibit per-video inference times on the order of milliseconds. This efficiency is largely attributed to their lightweight architectures and minimal memory usage, often requiring only hundreds of megabytes of GPU memory. In contrast, Video-HLM incurs substantially higher inference time due to its large model size and corresponding memory demand, requiring about 24 GB GPU memory.

Our quantized QuFiH model demonstrates a favorable trade-off: it achieves slightly lower inference latency than Video-HLM while consuming significantly less GPU memory. Although both Video-HLM and QuFiH are relatively much slower compared to lightweight baselines, their average inference time per video remains within 60–70 ms—still comfortably below the 1-s threshold—making them reasonably fast for practical use.

Importantly, there remains substantial room for improving the inference efficiency of QuFiH. The current implementation is entirely software-based and thus suboptimal. If future hardware architectures provide native support for 2-bit data types and arithmetic operations, the overhead from bitwise dequantization could be effectively eliminated. Such hardware-level support would not only reduce memory consumption further but also accelerate inference and enable broader deployment of 2-bit quantized models in real-world applications.

In summary, the development of native 2-bit computation and storage capabilities—both in software and hardware—presents a promising yet challenging research direction. It warrants continued attention and investment from both academia and industry, with potential benefits for a wide range of resource-constrained AI systems.

4.9. t-SNE visualization

We randomly selected 10 categories from the ActivityNet and FCVID datasets, with 50 samples per category, and used t-SNE to project them into a 2D space for visualization, as shown in Figs. 4 and 5. The experimental results demonstrate that our method significantly outperforms other competing methods (or at least is comparable with the most competitive AVHash method) in clustering performance, as detailed below:

1. Comparison with Supervised Methods. Our method achieves better clustering clarity compared to supervised methods such as AVH and DSVH. While these methods exhibit some degree of clustering capability for certain categories, they show considerable overlap at category boundaries, indicating limited discriminative power for video features. In contrast, our method separates different categories more distinctly, demonstrating stronger feature representation capability.

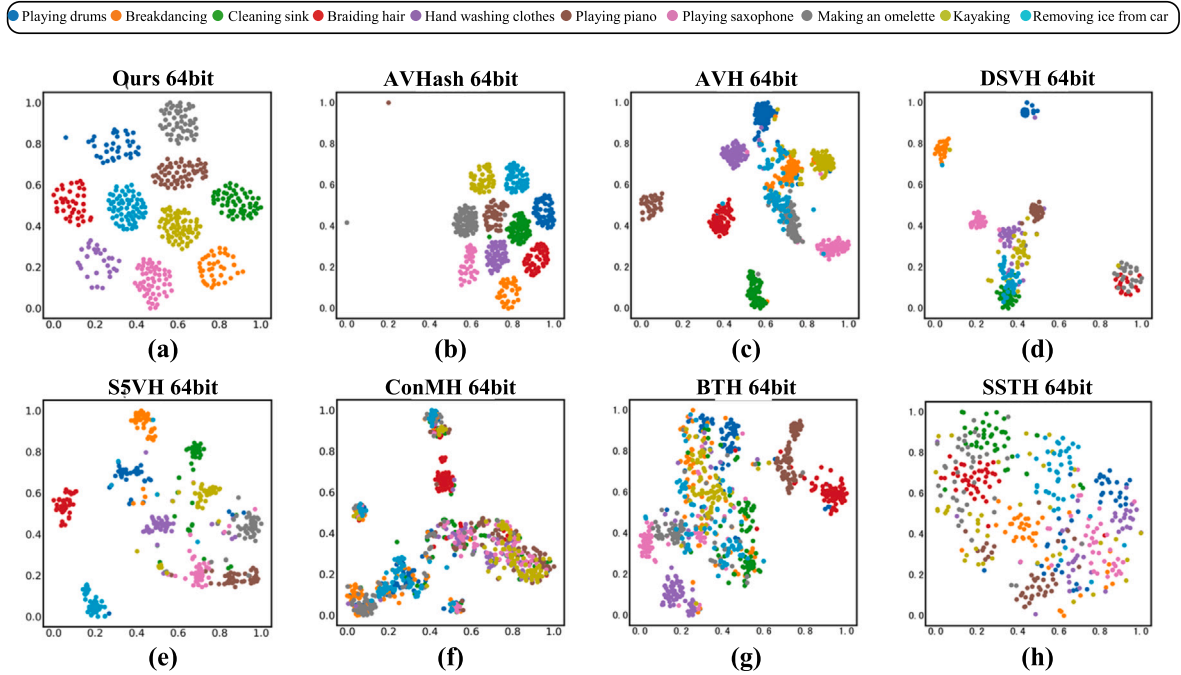


Fig. 5. t-SNE visualizations of QuFiH and 7 most competitive methods on FCVID dataset.

2. Comparison with Unsupervised Methods. Our method also outperforms unsupervised methods such as ConMH and BTH. These methods exhibit a high category confusion rate during clustering, especially for visually or semantically similar categories, leading to poor clustering performance. In contrast, our method effectively reduces category overlap through efficient feature extraction and quantization strategies, significantly improving clustering accuracy.

3. Comparison with the Current State-of-the-Art Method. Our method performs on par with the state-of-the-art AVHash method, and even surpasses it in certain categories (Fig. 5(a) vs. Fig. 5(b)). This result indicates that, despite adopting hybrid low-bit quantization and block-level parameter-efficient fine-tuning strategies, our method maintains strong feature representation capability. Furthermore, our method offers significant advantages in computational efficiency and storage cost, further highlighting its practical potential for real-world applications.

5. Case studies: Video retrieval simulations

To evaluate the retrieval performance of our method compared to other baseline methods, we randomly selected one video from each dataset as the query video. Based on the Hamming distance between the query video and the candidate video database, we retrieved the top 20 most relevant videos.

Fig. 6 presents two query examples: “Playing bagpipe” from ActivityNet (Fig. 6(a)) and “Beatbox” from FCVID (Fig. 6(b)). The corresponding retrieval results are shown in Figs. 7 and 8, respectively. In the visualization of retrieval results, green borders indicate correctly matched videos belonging to the query category, while red borders indicate incorrectly matched videos. Additionally, the notation “A/B” indicates that among the top B (=20) returned videos, A represents the number of correctly retrieved results. In Figs. 7 and 8, the first five rows display retrieval results from supervised methods, while the last five rows show results from unsupervised methods. Overall, supervised methods significantly outperform unsupervised methods, showing better performance in the number of correctly retrieved videos. Furthermore, our method demonstrates excellent performance on both datasets, consistently surpassing both supervised and unsupervised baseline methods. Notably, in these two examples, both our method and AVHash achieved perfect retrieval scores (20/20).

Specifically, in Fig. 7, our method accurately identifies “Playing bagpipe” across various backgrounds, demonstrating performance equivalent to AVHash. Other methods perform poorly when dealing with activities that are visually similar but acoustically distinct. For example, DSVH and SRH frequently misidentify “Playing accordion” and “Playing saxophone” as “Playing bagpipes”. This confusion does not arise from a lack of visual cues, as the scenes and even the instrument appearances in these videos are very similar to the query video. Rather, it stems from these methods’ inability to effectively distinguish crucial audio differences.

Similarly, in Fig. 8, our method again achieves 100% retrieval accuracy in the top 20 results (20/20), matching AVHash’s performance. However, methods relying solely on visual features (such as AVH, DSVH, and SRH) perform poorly in distinguishing “Beatbox” from visually similar activities like “SingingOnStage”, “RockBandPerformance”, and “WeddingReception”. Notably, while these videos all contain stage performance scenes, they feature different types of performances that are difficult to differentiate

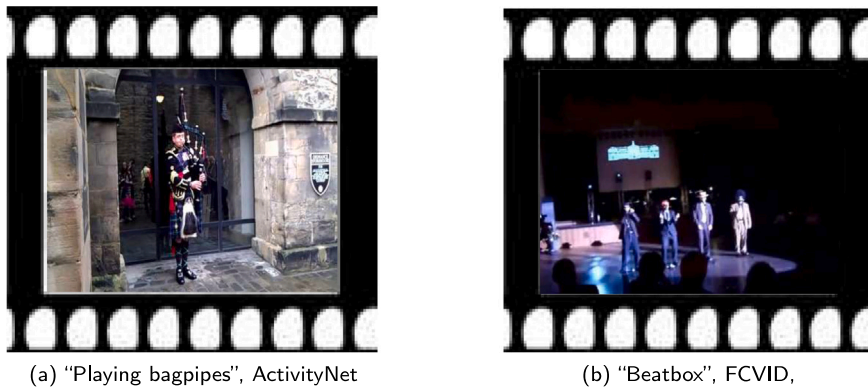


Fig. 6. Two randomly selected queries (i.e., Figs. 6(a), and 6(b)) from ActivityNet and FCVID datasets, respectively. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

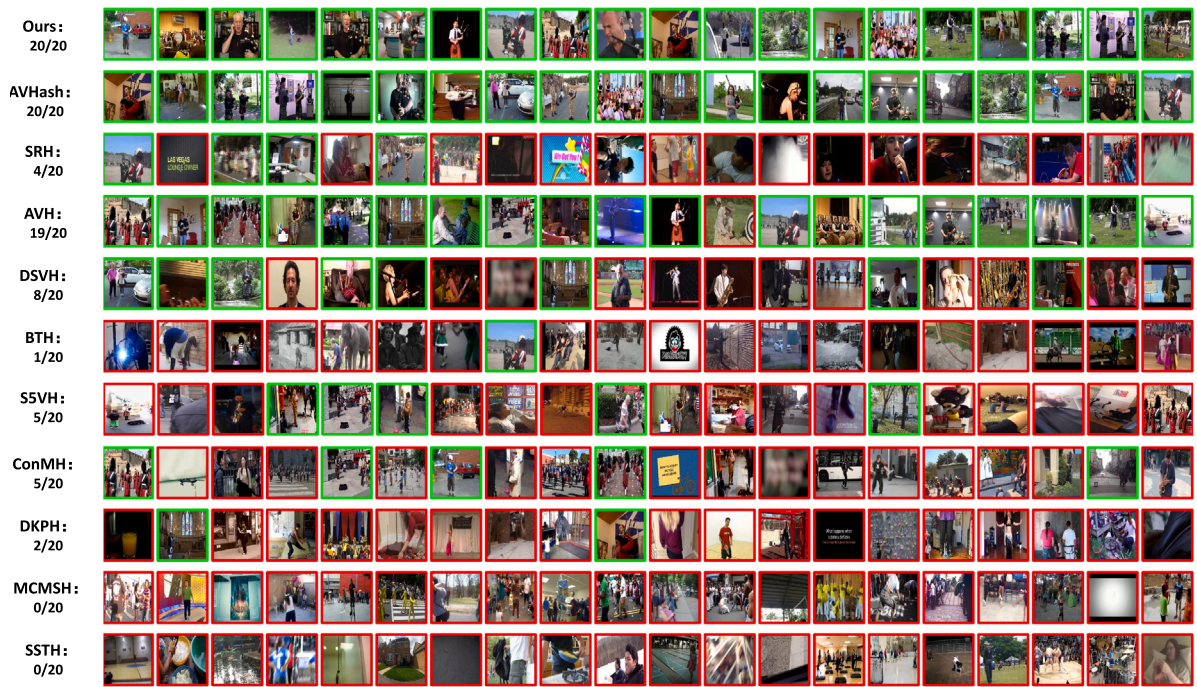


Fig. 7. The top-20 retrieved videos of different methods for the query “Playing bagpipes” (i.e., Fig. 6(a)) from ActivityNet. Note that the bounding boxes in green denote the correct results, while those in red represent the wrong ones. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

accurately without the aid of audio signals. Unsupervised methods (such as DKPH and SSTH) perform even worse, further exposing their limitations in understanding video semantic content. These methods often over-emphasize irrelevant background details (such as “dim lighting”) while failing to capture the essential characteristics of the core activities in the videos, indicating significant deficiencies in their video semantic understanding capabilities.

6. Discussions and implications

6.1. Discussions of results

The proposed **QuFiH** framework demonstrates significant advancements in video hashing by integrating **hybrid low-bit quantization** and **block-level parameter-efficient fine-tuning** (e.g., LoRA and Propulsion). The experimental results validate its superiority in **computational efficiency, storage cost, and retrieval performance** compared to existing methods.

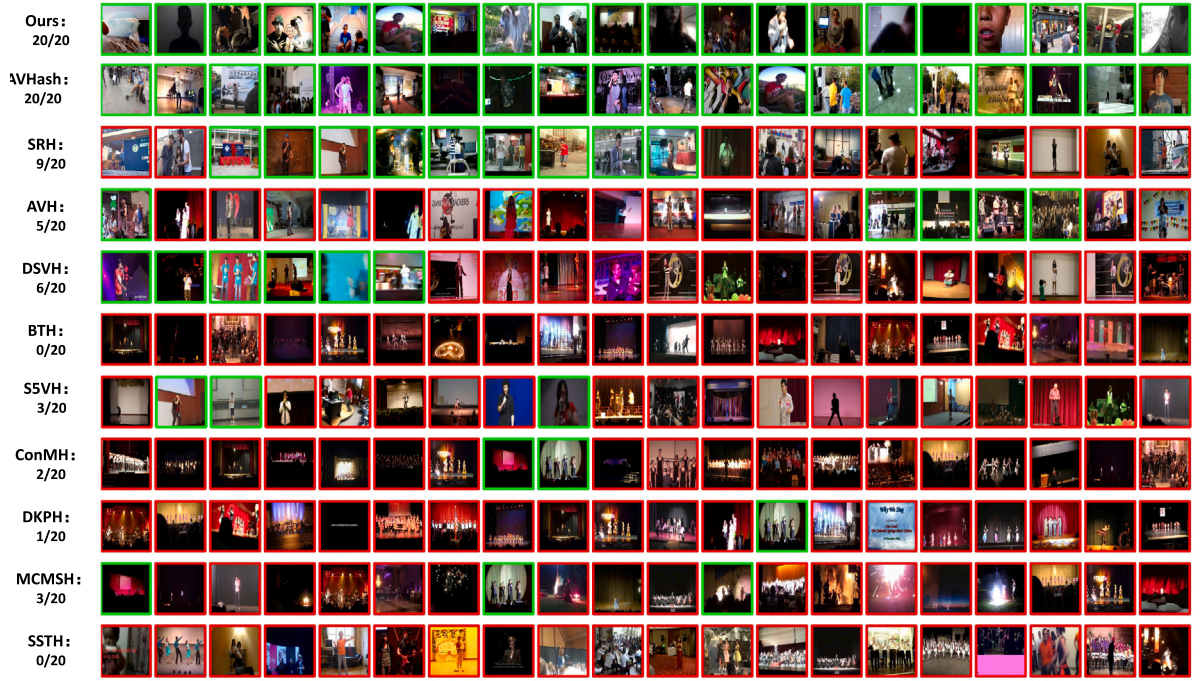


Fig. 8. The top-20 retrieved videos of different methods for the query “Beatbox” (i.e., Fig. 6(b)) from FCVID. Note that the bounding boxes in green denote the correct results, while those in red represent the wrong ones. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

1. Efficiency-Accuracy Trade-off

- The hybrid low-bit quantization strategy effectively reduces model size while maintaining high retrieval accuracy, proving that aggressive quantization does not necessarily degrade performance when combined with proper fine-tuning techniques.
- The block-level LoRA and Propulsion methods, though slightly less performant than traditional linear-level fine-tuning, drastically reduce trainable parameters, making them highly suitable for large-scale video retrieval.

2. Generalization and Transferability

- The integration of knowledge distillation with downstream task fine-tuning improves generalization, enabling QuFiH to adapt across diverse retrieval scenarios. This highlights the potential of coupling pre-trained large models with task-oriented optimization for video hashing.

3. Comparison with State-of-the-Art

- QuFiH outperforms existing hashing techniques in mAP, even surpassing the full-precision Video-HLM in some cases. This indicates that quantization-aware training and efficient fine-tuning can mitigate accuracy loss typically associated with model compression.

Overall, QuFiH demonstrates that hybrid quantization, block-level fine-tuning, and synergistic knowledge transfer can jointly deliver efficient yet accurate video retrieval, offering a promising paradigm for deploying large models in practical, resource-constrained environments.

6.2. Theoretical contributions

This work makes several theoretical contributions to the fields of video hashing, model quantization, and parameter-efficient adaptation of large models:

- **Foundation of Video-Hashing Large Models (Video-HLM).** We are the first to conceptualize and formalize the notion of a video hashing large model (Video-HLM), extending the paradigm of pre-trained multi-modal large models (e.g., Video-LLaMA) to the task of video hashing. This establishes a theoretical framework for integrating large-scale cross-modal pre-training with compact binary code generation, providing a new perspective for bridging semantic representation learning and efficient video retrieval.

- **Hybrid Low-Bit Quantization Principle.** We introduce a theoretical rationale for heterogeneous quantization across model components, grounded in the differential functional roles of the foundation model and the hash model. Specifically, we argue that lower layers engaged in fine-grained feature extraction are more sensitive to quantization noise and therefore require higher precision, whereas higher layers performing semantic abstraction are more tolerant of reduced precision. This principle, inspired by cognitive theories of human visual processing, motivates our hybrid low-bit quantization strategy (4-bit for foundation model and 2-bit for hash model), which formalizes a new balance between efficiency and expressiveness.
- **Block-Level Parameter-Efficient Fine-Tuning.** We provide the first systematic theoretical analysis of block-level parameter-efficient fine-tuning, contrasting it with the widely adopted linear-level strategies. By applying LoRA or Propulsion directly at the Transformer block granularity, we demonstrate how parameter redundancy can be minimized while preserving representational capacity. This reframes parameter-efficient tuning as an optimization problem over block structures rather than isolated linear transformations, offering a more principled pathway toward scalable adaptation.
- **Synergistic Integration of Distillation and Fine-Tuning.** We advance the theory of model compression and transfer learning by examining the interaction between knowledge distillation and downstream task fine-tuning. While both methods are individually well-studied, their joint optimization has not been rigorously investigated. We theoretically justify and empirically validate a “distill-then-finetune” pipeline, showing that sequentially aligning student representations before task-specific optimization leads to more stable and accurate hash code learning.

Taken together, these contributions establish a theoretical foundation for **efficient and high-precision video hashing with large models**, linking ideas from quantization theory, cognitive-inspired design, and parameter-efficient adaptation. They also open new avenues for future research on model compression strategies tailored to multi-modal retrieval tasks.

6.3. Practical implications

The proposed QuFiH framework carries several practical implications for the deployment of large models in video retrieval and related applications:

- **Deployment in Resource-Constrained Environments.** QuFiH achieves up to a 6.84× compression rate while maintaining or even surpassing the retrieval accuracy of the full-precision Video-HLM. This efficiency–accuracy balance demonstrates its suitability for mobile, embedded, and edge platforms, where memory and compute resources are limited but retrieval accuracy remains critical.
- **Scalability for Large-Scale Video Platforms.** Experimental results on large benchmark datasets indicate that QuFiH not only reduces storage overhead but also consistently delivers superior retrieval performance compared to existing baselines. Notably, under the mAP@100 metric, QuFiH surpasses the full-precision Video-HLM while maintaining a substantial compression rate. These advantages highlight QuFiH’s applicability to large-scale real-world scenarios such as social media recommendation systems, surveillance networks, and medical video archives, where efficient yet accurate retrieval is essential to cope with rapidly expanding data volumes.
- **Domain Adaptation with Limited Data.** The proposed distill-then-finetune pipeline enhances robustness across datasets. In transfer experiments, QuFiH achieves higher mAP with fewer annotations, confirming its value in domains such as education and healthcare, where labeled video data are often scarce.
- **Extensibility to Multimodal Retrieval.** Although designed for video hashing, the core principles of QuFiH—hybrid quantization, block-level fine-tuning, and synergistic distillation—extend naturally to cross-modal retrieval tasks (e.g., video–text or audio–video). When integrated with industrial-scale retrieval infrastructures such as vector databases (e.g., FAISS, Milvus), QuFiH provides a scalable and practical foundation for multimodal content indexing.

In summary, QuFiH not only advances the theoretical understanding of quantization and parameter-efficient tuning but also provides a concrete pathway for building **efficient, accurate, and deployable video retrieval systems** across multiple real-world domains.

7. Conclusions

The QuFiH framework successfully addresses key challenges in video hashing by:

- Reducing computational and storage costs through hybrid low-bit quantization while maintaining high retrieval accuracy.
- Enhancing parameter efficiency via block-level LoRA and Propulsion, making large-scale video retrieval more feasible.
- Improving generalization through knowledge distillation and downstream fine-tuning, ensuring adaptability to diverse video retrieval tasks.

These innovations establish QuFiH as a practical and scalable solution for efficient video retrieval, setting a new benchmark for future research in video hashing.

Building on these findings, several promising directions remain open:

- *Dynamic Quantization Strategies* — exploring adaptive bit-width allocation across model layers to further optimize the trade-off between efficiency and accuracy;

- **Hardware-Aware Optimization** — developing FPGA/ASIC-friendly implementations to achieve maximal inference speed and energy efficiency;
- **Integration with Vision-Language Models (VLMs)** — extending QuFiH to multimodal hashing (e.g., CLIP-based retrieval) for richer semantic alignment and cross-modal retrieval capabilities.

Advancing along these directions will enable QuFiH to evolve into a versatile, robust, and future-ready framework for next-generation video retrieval systems.

CRedit authorship contribution statement

Fudong Li: Writing – original draft, Validation, Methodology, Data curation, Conceptualization. **Yuxiang Zhou:** Writing – original draft, Validation, Methodology, Investigation, Data curation. **Qinglai Yang:** Visualization, Validation, Investigation. **Yong Chen:** Writing – review & editing, Supervision, Methodology, Investigation, Conceptualization. **Dell Zhang:** Writing – review & editing, Supervision, Investigation, Conceptualization. **Xuelong Li:** Writing – review & editing, Supervision, Conceptualization.

Declaration of Generative AI and AI-assisted technologies in the writing process

During the preparation of this manuscript, the authors employed ChatGPT (<https://chatgpt.com/>) to assist in enhancing the clarity and readability of the text. All content generated with this tool was subsequently reviewed and revised by the authors as necessary. The authors assume full responsibility for the final version of the article.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work is supported in part by National Natural Science Foundation of China (NSFC, Grant No. 62372054, 62006005) and National Key Research and Development Program of China (Grant No. 2022YFC3302200).

Data availability

Data is now available at <https://github.com/kydbj/QuFiH>.

References

- Beyer, L., Zhai, X., Royer, A., Markeeva, L., Anil, R., & Kolesnikov, A. (2022). Knowledge distillation: A good teacher is patient and consistent. In *IEEE/CVF conference on computer vision and pattern recognition, CVPR 2022, New Orleans, LA, USA, June 18-24, 2022* (pp. 10915–10924).
- Chen, L. (2023). Exploring the impact of short videos on society and culture: An analysis of social dynamics and cultural expression. *Pacific International Journal*, 6, 115–118.
- Chen, Y., Hou, Y., Leng, S., Zhang, Q., Lin, Z., & Zhang, D. (2021). Long-tail hashing. In *SIGIR '21: the 44th international ACM SIGIR conference on research and development in information retrieval, virtual event, Canada, July 11-15, 2021* (pp. 1328–1338).
- Chen, H., Hu, C., Lee, F., Lin, C., Yao, W., Chen, L., et al. (2021). A supervised video hashing method based on a deep 3D convolutional neural network for large-scale video retrieval. *Sensors*, 21(9), 3094.
- Chen, Y., Tian, Z., Zhang, H., Wang, J., & Zhang, D. (2020). Strongly constrained discrete hashing. *IEEE Transactions on Image Processing*, 29, 3596–3611.
- Chen, Y., Zhou, Y., Dong, H., Liu, R., Lin, Z., & Zhang, D. (2025). AV-NAS: Audio-visual multi-level semantic neural architecture search for video hashing. In *ACMMM* (pp. 895–905). ACM.
- Daniel, R., McKechnie, T., Kruse, C. C., Levin, M., Lee, Y., Doumouas, A. G., et al. (2023). Video-based coaching for surgical residents: A systematic review and meta-analysis. *Surgical Endoscopy*, 37, 1429–1439.
- Datar, M., Immorlica, N., Indyk, P., & Mirrokni, V. S. (2004). Locality-sensitive hashing scheme based on p-stable distributions. In *Proceedings of the 20th ACM symposium on computational geometry, Brooklyn, New York, USA, June 8-11, 2004* (pp. 253–262).
- Dettmers, T., Lewis, M., Belkada, Y., & Zettlemoyer, L. (2022). GPT3.int8(): 8-bit matrix multiplication for transformers at scale. In *Advances in neural information processing systems 35: annual conference on neural information processing systems 2022, neurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*.
- Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2023). Qlora: Efficient finetuning of quantized LLMs. In *Advances in neural information processing systems 36: annual conference on neural information processing systems 2023, neurIPS 2023, New Orleans, LA, USA, December 10 - 16, 2023*.
- Ding, N., Qin, Y., Yang, G., Wei, F., Yang, Z., Su, Y., et al. (2023). Parameter-efficient fine-tuning of large-scale pre-trained language models. *Nature Machine Intelligence*, 5(3), 220–235.
- Dong, Z., Liu, X., Chen, B., Polak, P., & Zhang, P. (2024). MuseChat: A conversational music recommendation system for videos. In *IEEE/CVF conference on computer vision and pattern recognition, CVPR 2024, Seattle, WA, USA, June 16-22, 2024* (pp. 12775–12785).
- Duan, J., Hao, Y., Zhu, B., Cheng, L., Zhou, P., & Wang, X. (2024). Efficient unsupervised video hashing with contextual modeling and structural controlling. *IEEE Transactions on Multimedia*, 26, 7438–7450.
- Duong, H.-T., Le, V.-T., & Hoang, V. T. (2023). Deep learning-based anomaly detection in video surveillance: A survey. *Sensors*, 5024.
- Fan, Z., Li, F., Liu, H., He, J., & Du, X. (2024). PoseRec: 3D human pose driven online advertisement recommendation for micro-videos. In *Proceedings of the 2024 international conference on multimedia retrieval, ICMR 2024, Phuket, Thailand, June 10-14, 2024* (pp. 37–45).

- Fang, Y., Wang, W., Xie, B., Sun, Q., Wu, L., Wang, X., et al. (2023). EVA: Exploring the limits of masked visual representation learning at scale. In *IEEE/CVF conference on computer vision and pattern recognition, CVPR 2023, Vancouver, BC, Canada, June 17-24, 2023* (pp. 19358–19369).
- Farhad, M., Masud, M. M., Beg, A., Ahmad, A., Ahmed, L. A., & Memon, S. (2023). Cardiac phase detection in echocardiography using convolutional neural networks. *Scientific Reports*, 13, 8908.
- Frantar, E., Ashkboos, S., Hoefler, T., & Alistarh, D. (2022). GPTQ: Accurate post-training quantization for generative pre-trained transformers. CoRR abs/2210.17323.
- Girdhar, R., El-Nouby, A., Liu, Z., Singh, M., Alwala, K. V., Joulin, A., et al. (2023). ImageBind one embedding space to bind them all. In *IEEE/CVF conference on computer vision and pattern recognition, CVPR 2023, Vancouver, BC, Canada, June 17-24, 2023* (pp. 15180–15190).
- GLM, T., Zeng, A., Xu, B., Wang, B., Zhang, C., Yin, D., et al. (2024). Chatglm: A family of large language models from glm-130b to glm-4 all tools. arXiv preprint arXiv:2406.12793.
- Gong, Y., Lazebnik, S., Gordo, A., & Perronnin, F. (2013). Iterative quantization: A procrustean approach to learning binary codes for large-scale image retrieval. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 35(12), 2916–2929.
- Gu, Y., Ma, C., & Yang, J. (2016). Supervised recurrent hashing for large scale video retrieval. In *Proceedings of the 2016 ACM conference on multimedia conference, MM 2016, Amsterdam, The Netherlands, October 15-19, 2016* (pp. 272–276).
- Hao, Y., Duan, J., Zhang, H., Zhu, B., Zhou, P., & He, X. (2022). Unsupervised video hashing with multi-granularity contextualization and multi-structure preservation. In *MM '22: the 30th ACM international conference on multimedia, Lisboa, Portugal, October 10 - 14, 2022* (pp. 3754–3763).
- Heilbron, F. C., Escorcia, V., Ghanem, B., & Niebles, J. C. (2015). ActivityNet: A large-scale video benchmark for human activity understanding. In *IEEE conference on computer vision and pattern recognition, CVPR 2015, Boston, MA, USA, June 7-12, 2015* (pp. 961–970).
- Hossain, M. A. F., & Zhu, F. (2024). Structured pruning and quantization for learned image compression. In *IEEE* (pp. 3730–3736). IEEE.
- Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., et al. (2022). LoRA: Low-rank adaptation of large language models. In *The tenth international conference on learning representations, ICLR 2022, virtual event, April 25-29, 2022*.
- Huang, W., Liu, Y., Qin, H., Li, Y., Zhang, S., Liu, X., et al. (2024). BiLLM: Pushing the limit of post-training quantization for LLMs. In *Forty-first international conference on machine learning, ICML 2024, Vienna, Austria, July 21-27, 2024*.
- Jiang, Y., Wu, Z., Wang, J., Xue, X., & Chang, S. (2018). Exploiting feature and class relationships in video categorization with regularized deep neural networks. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 40(2), 352–364.
- Kang, W., Li, W., & Zhou, Z. (2016). Column sampling based discrete supervised hashing. In *Proceedings of the thirtieth AAAI conference on artificial intelligence, February 12-17, 2016, Phoenix, Arizona, USA* (pp. 1230–1236).
- Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. In *3rd international conference on learning representations, ICLR 2015, San Diego, CA, USA, May 7-9, 2015, conference track proceedings*.
- Kowsher, M., Prottasha, N. J., & Bhat, P. (2025). Propulsion: Steering LLM with tiny fine-tuning. In *Proceedings of the 31st international conference on computational linguistics, COLING 2025, Abu Dhabi, UAE, January 19-24, 2025* (pp. 7569–7597).
- Li, S., Chen, Z., Lu, J., Li, X., & Zhou, J. (2019). Neighborhood preserving hashing for scalable video retrieval. In *IEEE/CVF international conference on computer vision, ICCV 2019, Seoul, Korea (south), October 27 - November 2, 2019* (pp. 8211–8220).
- Li, S., Li, X., Lu, J., & Zhou, J. (2021). Self-supervised video hashing via bidirectional transformers. In *IEEE conference on computer vision and pattern recognition, CVPR 2021, virtual, June 19-25, 2021* (pp. 13549–13558).
- Li, Z., Li, H., & Meng, L. (2023). Model compression for deep neural networks: A survey. *Computers*, 12(3), 60.
- Li, J., Li, D., Savarese, S., & Hoi, S. C. H. (2023). BLIP-2: bootstrapping language-image pre-training with frozen image encoders and large language models. In *Proceedings of machine learning research: Vol. 202, International conference on machine learning, ICML 2023, 23-29 July 2023, Honolulu, Hawaii, USA* (pp. 19730–19742).
- Li, Q., Sun, Z., He, R., & Tan, T. (2017). Deep supervised discrete hashing. In *Advances in neural information processing systems 30: annual conference on neural information processing systems 2017, December 4-9, 2017, Long Beach, CA, USA* (pp. 2482–2491).
- Li, Q., Tian, X., & Ng, W. W. Y. (2024). Self-supervised temporal sensitive hashing for video retrieval. *IEEE Transactions on Multimedia*, 26, 9021–9035.
- Li, P., Xie, H., Ge, J., Zhang, L., Min, S., & Zhang, Y. (2022). Dual-stream knowledge-preserving hashing for unsupervised video retrieval. In *Lecture notes in computer science: Vol. 13674, Computer vision - ECCV 2022 - 17th European conference, Tel Aviv, Israel, October 23-27, 2022, proceedings, part XIV* (pp. 181–197).
- Lin, J., Tang, J., Tang, H., Yang, S., Xiao, G., & Han, S. (2024). AWQ: Activation-aware weight quantization for on-device LLM compression and acceleration. *GetMobile: Mobile Computing and Communications*, 28(4), 12–17.
- Liong, V. E., Lu, J., Tan, Y., & Zhou, J. (2017). Deep video hashing. *IEEE Transactions on Multimedia*, 19(6), 1209–1219.
- Liu, H.-I., Galindo, M., Xie, H., Wong, L.-K., Shuai, H.-H., Li, Y.-H., et al. (2024). Lightweight deep learning for resource-constrained environments: A survey. *ACM Computing Surveys*, 56, 1–42.
- Liu, Z., Oguz, B., Zhao, C., Chang, E., Stock, P., Mehdad, Y., et al. (2024). LLM-QAT: Data-free quantization aware training for large language models. In *Findings of the association for computational linguistics, ACL 2024, Bangkok, Thailand and virtual meeting, August 11-16, 2024* (pp. 467–484).
- Pan, J., Lin, Z., Zhu, X., Shao, J., & Li, H. (2022). ST-adapter: Parameter-efficient image-to-video transfer learning. In *Advances in neural information processing systems 35: annual conference on neural information processing systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*.
- Park, S., & Moon, S. (2025). CURING large models: Compression via CUR decomposition. CoRR abs/2501.04211.
- Pu, R., Sun, Y., Qin, Y., Ren, Z., Song, X., Zheng, H., et al. (2025). Robust self-paced hashing for cross-modal retrieval with noisy labels. In *AAAI* (pp. 19969–19977). AAAI Press.
- Shen, F., Shen, C., Liu, W., & Shen, H. T. (2015). Supervised discrete hashing. In *IEEE conference on computer vision and pattern recognition, CVPR 2015, Boston, MA, USA, June 7-12, 2015* (pp. 37–45).
- Song, J., Yang, Y., Huang, Z., Shen, H. T., & Hong, R. (2011). Multiple feature hashing for real-time large scale near-duplicate video retrieval. In *Proceedings of the 19th international conference on multimedia 2011, Scottsdale, AZ, USA, November 28 - December 1, 2011* (pp. 423–432).
- Sun, Y., Dai, J., Ren, Z., Chen, Y., Peng, D., & Hu, P. (2024). Dual self-paced cross-modal hashing. In *AAAI* (pp. 15184–15192). AAAI Press.
- Sun, Y., Ren, Z., Hu, P., Peng, D., & Wang, X. (2024). Hierarchical consensus hashing for cross-modal retrieval. *IEEE Transactions on Multimedia*, 26, 824–836.
- Sung, Y., Cho, J., & Bansal, M. (2022). VL-ADAPTER: Parameter-efficient transfer learning for vision-and-language tasks. In *IEEE/CVF conference on computer vision and pattern recognition, CVPR 2022, New Orleans, LA, USA, June 18-24, 2022* (pp. 5217–5227).
- Tolstikhin, I. O., Houlsby, N., Kolesnikov, A., Beyer, L., Zhai, X., Unterthiner, T., et al. (2021). Mlp-mixer: An all-mlp architecture for vision. vol. 34, In *Advances in neural information processing systems 34: annual conference on neural information processing systems 2021, NeurIPS 2021, December 6-14, 2021, Virtual* (pp. 24261–24272).
- Wang, J., Lian, N., Li, J., Wang, Y., Feng, Y., Chen, B., et al. (2025). Efficient self-supervised video hashing with selective state spaces. In *AAAI* (pp. 7753–7761).
- Wang, Y., Nie, X., Shi, Y., Zhou, X., & Yin, Y. (2021). Attention-based video hashing for large-scale video retrieval. *IEEE Transactions on Cognitive and Developmental Systems*, 13(3), 491–502.
- Wang, L., Pan, Y., Liu, C., Lai, H., Yin, J., & Liu, Y. (2023). Deep hashing with minimal-distance-separated hash centers. In *IEEE/CVF conference on computer vision and pattern recognition, CVPR 2023, Vancouver, BC, Canada, June 17-24, 2023* (pp. 23455–23464).
- Wang, Z., Panda, R., Karlinsky, L., Feris, R., Sun, H., & Kim, Y. (2023). Multitask prompt tuning enables parameter-efficient transfer learning. In *The eleventh international conference on learning representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net.

- Wang, Y., Wang, J., Chen, B., Zeng, Z., & Xia, S. (2023). Contrastive masked autoencoders for self-supervised video hashing. In *Thirty-seventh AAAI conference on artificial intelligence, AAAI 2023, thirty-fifth conference on innovative applications of artificial intelligence, IAAI 2023, thirteenth symposium on educational advances in artificial intelligence, EAAI 2023, Washington, DC, USA, February 7-14, 2023* (pp. 2733–2741).
- Wei, R., Liu, Y., Song, J., Cui, H., Xie, Y., & Zhou, K. (2023). CHAIN: Exploring global-local spatio-temporal information for improved self-supervised video hashing. In *Proceedings of the 31st ACM international conference on multimedia, MM 2023, Ottawa, ON, Canada, 29 October 2023- 3 November 2023* (pp. 1677–1688).
- Xia, K., Ma, Y., Liu, X., Mu, Y., & Liu, L. (2017). Temporal binary coding for large-scale video search. In *Proceedings of the 2017 ACM on multimedia conference, MM 2017, mountain view, CA, USA, October 23-27, 2017* (pp. 333–341).
- Xiao, G., Lin, J., Seznec, M., Wu, H., Demouth, J., & Han, S. (2023). SmoothQuant: Accurate and efficient post-training quantization for large language models. vol. 202, In *International conference on machine learning, ICML 2023, 23-29 July 2023, Honolulu, Hawaii, USA* (pp. 38087–38099).
- Xu, Z. (2017). *Large-scale video analysis and understanding* [Ph.D. thesis], University of Technology Sydney (Australia).
- Xu, C., & McAuley, J. J. (2023). A survey on model compression and acceleration for pretrained language models. In *Thirty-seventh AAAI conference on artificial intelligence, AAAI 2023, thirty-fifth conference on innovative applications of artificial intelligence, IAAI 2023, thirteenth symposium on educational advances in artificial intelligence, EAAI 2023, Washington, DC, USA, February 7-14, 2023* (pp. 10566–10575).
- Yang, Z., Li, Z., Jiang, X., Gong, Y., Yuan, Z., Zhao, D., et al. (2022). Focal and global knowledge distillation for detectors. In *IEEE/CVF conference on computer vision and pattern recognition, CVPR 2022, New Orleans, LA, USA, June 18-24, 2022* (pp. 4633–4642).
- Yang, C., Liu, C., Deng, X., Kim, D., Mei, X., Shen, X., et al. (2024). 1.58-Bit FLUX. arXiv preprint arXiv:2412.18653.
- Yang, Z., Liu, J., Wu, Z., Wu, P., & Liu, X. (2023). Video event restoration based on keyframes for video anomaly detection. In *IEEE/CVF conference on computer vision and pattern recognition, CVPR 2023, Vancouver, BC, Canada, June 17-24, 2023* (pp. 14592–14601).
- Yao, Z., Aminabadi, R. Y., Zhang, M., Wu, X., Li, C., & He, Y. (2022). ZeroQuant: Efficient and affordable post-training quantization for large-scale transformers. In *Advances in neural information processing systems 35: annual conference on neural information processing systems 2022, neurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*.
- Ye, G., Liu, D., Wang, J., & Chang, S. (2013). Large-scale video hashing via structure learning. In *IEEE international conference on computer vision, ICCV 2013, Sydney, Australia, December 1-8, 2013* (pp. 2272–2279).
- Yuan, Z., Shang, Y., & Dong, Z. (2024). PB-LLM: Partially binarized large language models. In *The twelfth international conference on learning representations, ICLR 2024, Vienna, Austria, May 7-11, 2024*.
- Yuan, L., Wang, T., Zhang, X., Tay, F. E. H., Jie, Z., Liu, W., et al. (2020). Central similarity quantization for efficient image and video retrieval. In *IEEE/CVF conference on computer vision and pattern recognition, CVPR 2020, Seattle, WA, USA, June 13-19, 2020* (pp. 3080–3089).
- Zhang, H., Li, X., & Bing, L. (2023). Video-llama: An instruction-tuned audio-visual language model for video understanding. arXiv preprint arXiv:2306.02858.
- Zhang, H., Wang, M., Hong, R., & Chua, T. (2016). Play and rewind: Optimizing binary representations of videos by self-supervised temporal hashing. In *Proceedings of the 2016 ACM conference on multimedia conference, MM 2016, Amsterdam, the Netherlands, October 15-19, 2016* (pp. 781–790).
- Zheng, Y., Chen, Y., Qian, B., Shi, X., Shu, Y., & Chen, J. (2024). A review on edge large language models: Design, execution, and applications. *ACM Computing Surveys*.
- Zhou, Y., Sun, Z., Liu, R., Chen, Y., & Zhang, D. (2024). AVHash: Joint audio-visual hashing for video retrieval. In *Proceedings of the 32nd ACM international conference on multimedia, MM 2024, Melbourne, VIC, Australia, 28 October 2024 - 1 November 2024* (pp. 2370–2378).
- Zhu, X., Li, J., Liu, Y., Ma, C., & Wang, W. (2024). A survey on model compression for large language models. *Transactions of the Association for Computational Linguistics*, 12, 1556–1577.